

تقرير تفصيلي للتحكم بال Shuttles وال Rail System

هذا التقرير يشرح خطوة بخطوة كيف يعمل نظام ال kinematic shuttles في Room 315, كيف نتحكم بال shuttles وال

switches وال stoppers وال sensors, وكيف أضفنا ال topics وال custom messages داخل ROS 2.

هذا الإصدار مكتوب لشخص لم يعمل سابقاً بـ ROS 2. كل مفهوم سيتم ربطه مباشرة بملف من المشروع وبمثال من الكود

وبأمر يمكن تجربته في Terminal.

وملفات الكود باللغة الإنكليزية.	Gazebo و	ROS	اللغة: العربية، مع إبقاء مصطلحات
---------------------------------	----------	-----	----------------------------------

الملفات الأساسية:

mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py•

mfja_rail_interfaces/msg/*.msg•

mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py•

mfja_room_315_bringup/launch/room_315_only.launch.py•

mfja_3rd_floor_bringup/launch/full_floor.launch.py•

الهدف من التقرير ليس فقط تشغيل الأوامر، بل فهم المبدأ حتى تستطيع تعديل الكود وإضافة features جديدة بثقة.

فهرس التقرير

1. الغرض من النظام

2. نظرة عامة على ROS 2

3. ما هو Node و Topic و Message

4. ما هو Service و Parameter و Launch

5. هيكل workspace وال packages

6. Build و Source

7. Architecture العامة

8. مبدأ kinematic simulation

CSV segments و Rail graph.9

Path backend.10

shuttle ال حالة.11

12. كيف تزداد قيمة s رياضياً

13. معادلات الانتقال بين segments

14. تأثير stoppers و collision avoidance على s

Lifecycle.15

Startup shuttles.16

Runtime add stopped/moving.17

ON/OFF/RESET/REMOVE.18

Switch control.19

20. تشرح أوامر `ros2 topic pub`

21. من أين يأتي message type مثل `mfja_rail_interfaces/msg/SwitchCommand`

22. قراءة ملفات `msg` و `ros2 interface show`

Stopper control.23

Sensors feedback.24

Device YAML.25

Gazebo services.26

27. أين نعدل بالكود

28. كيف نضيف topic جديد

29. كيف نضيف message جديد

30. Troubleshooting و Debugging

31. Checklist نهائية

الفكرة العامة للمشروع

المشروع يمثل simulation لغرفة Room 315 ضمن طابق MFJA الثالث. لدينا rails ثابتة داخل Gazebo، و shuttles تتحرك فوقها، لكن الحركة ليست مبنية على فيزياء عجالات واحتكاك. الحركة مبنية على حساب موضع shuttle على path معروف، ثم إرسال pose إلى Gazebo.

المبدأ: الكود يحسب أين يجب أن يكون ال shuttle، ثم يحرك model في Gazebo عبر /

.world/<world_name>/set_pose

هذا يجعل النظام مناسباً للتجارب التعليمية والتحكم المنطقي:

- إضافة shuttle أثناء التشغيل.
- تشغيل وإيقاف shuttle بدون إعادة تشغيل Gazebo.
- اختبار switches و stoppers و sensors.
- اختبار collision avoidance بين shuttles.
- توصيل النظام مع robots في Gazebo.

أساسيات ROS 2: الفكرة قبل الكود

ROS 2 هو framework يربط أجزاء robot system مع بعضها. بدل أن يكون لدينا برنامج واحد ضخم، نقسم النظام إلى

nodes صغيرة تتواصل عبر topics و services و parameters.

المفهوم	المعنى	مثال من مشروعنا
Node	برنامج صغير مستقل	room_315_kinematic_shuttle_node.py
Topic	قناة نشر/استقبال رسائل	room_315/rails/right/shuttles/command/
Message	شكل البيانات المرسل على topic	mfja_rail_interfaces/msg/ShuttleCommand
Service	طلب/جواب	world/room_315_only/create/
Parameter	إعداد قابل للتغيير	speed, start_enabled
Launch	ملف يبدأ عدة nodes معاً	room_315_only.launch.py

Message و Topic و Node

عندما ترسل أمر مثل ON، أنت لا تستدعي function مباشرة. أنت تنشر رسالة على topic. ال node المشترك بهذا topic يستقبل الرسالة ويقرر ماذا يفعل.

Terminal ros2 topic pub Topic /room_315/rails/right/... Node callback

لذلك عند إضافة feature جديدة غالباً نقرر: هل نحتاج topic جديد؟ أم يكفي توسيع command على topic موجود؟ في feature الأخيرة اخترنا توسيع command داخل ShuttleCommand بدون إنشاء message جديد.

Launch و Parameter و Service

في ROS 2، ال topic جيد للأوامر المستمرة أو events. أما ال service فيستخدم عندما نحتاج request/response.

Gazebo create/remove/set_pose هي services لأننا نطلب من Gazebo تنفيذ عملية محددة.

النوع	متى نستخدمه؟	مثال
Topic	أمر أو state متكرر	shuttle command/state
Service	طلب يحتاج رد	Gazebo spawn/delete/set_pose في
Parameter	إعداد node	speed, shuttle_count
Launch	تشغيل مجموعة nodes مع arguments	Room 315 launch

مثال: عند إضافة shuttle جديد، node ينشئ object داخلي، ثم يستخدم Gazebo service

/world/<world>/create إذا model غير موجود مسبقاً.

هيكل ال Workspace وال Packages

المشروع meta-repository يحوي عدة ROS packages:

الدور	Package
تعريف custom messages الخاصة بال rails.	mfja_rail_interfaces
node الرئيسي، launch base, scripts, YAML configs	mfja_robot_control_config
models, worlds, meshes, URDF/SDF	mfja_3rd_floor_description
Room 315 launch entry point فقط.	mfja_room_315_bringup
launch entry point للطابق الكامل.	mfja_3rd_floor_bringup

الدور	Package
.top-level launch wrappers	mfja_3rd_floor_gz
عندما تضيف message جديد، غالباً تعدل mfja_rail_interfaces. عندما تضيف behavior جديد، غالباً تعدل .room_315_kinematic_shuttle_node.py	

Source و Build

بعد تعديل كود Python أو launch غالباً يكفي build عادي، وبعد تعديل messages يجب rebuild لل interface package أيضاً.

```
cd /home/tiago/ALI_ros2_ws
```

```
source /opt/ros/jazzy/setup.bash
```

```
colcon build --symlink-install
```

```
source install/setup.bash
```

إذا فتحت terminal جديد فقط:

```
cd /home/tiago/ALI_ros2_ws
```

```
source /opt/ros/jazzy/setup.bash
```

```
source install/setup.bash
```

لأن room_315_kinematic_shuttle_node.py installed ≤ program داخل package، استخدام ---symlink

install يساعدك ترى التعديلات بسرعة بدون نسخ يدوي.

Architecture العامة للنظام

الصورة التالية تلخص كيف تتحرك البيانات من المستخدم إلى Gazebo:

User Terminal ros2 topic pub room_315_kinematic_shuttle_node callbacks + kinematic core

Gazebo create / set_pose / remove YAML Devices slots, sensors, stoppers Rail Network

segments + switches State Topics shuttle/switch/sensor

مبدأ Kinematic Simulation

في simulation الفيزيائي التقليدي، نترك ال physics engine يحسب حركة الجسم من forces و joints و contacts.

هنا اخترنا kinematic simulation: نحن نحسب pose مباشرة.

Kinematic-first في مشروعنا

Physics-based

يعتمد على wheels/contacts/friction يعتمد على rail path معروف

أصعب للتحكم الدقيق أسهل لتجارب switch routing

يتأثر بال physics instability أكثر deterministic

واقعي أكثر ميكانيكياً مناسب للتحكم والمنطق والتوثيق

ال node يقوم كل tick بما يلي: حساب pose، نشر state، إرسال set_pose إلى Gazebo عند الحاجة.

CSV Segments و Rail Graph

ال rail ليست خطأً واحداً. هي graph من segments. كل segment له اسم مثل A23, A14, A12E, A12I. ال

switches تحدد أي successor سيختاره ال shuttle عند نهاية segment.

CSV geometry

الملفات داخل	raw_segments	و normalized_segments	تعطي points	لل rail
--------------	--------------	-----------------------	-------------	---------

Rail network YAML

rail_network_right.yaml	و rail_network_left.yaml	يحددان graph	وال switches
-------------------------	--------------------------	--------------	--------------

عندما يتغير switch من EXTERIOR إلى INTERIOR، يتغير المسار التالي من branch إلى آخر.

Path Backend: لماذا Cubic Hermite؟

ال CSV points تعطينا نقاطاً منفصلة. كي يتحرك shuttle بسلاسة، نحتاج path يمكن sampling عليه حسب arc

length. لذلك نستخدم cubic_hermite $\leq$ default.

الاستخدام

Backend

الحركة الطبيعية السلسلة.

cubic_hermite

Debug polyline ومقارنة مباشرة مع CSV.

```
# Normal mode
```

```
-p path_backend:=cubic_hermite
```

```
# Debug mode
```

```
-p path_backend:=polyline
```

حالة ال Shuttle

كل shuttle له state داخلي و state منشور على topic.

معنى	Field
اسم shuttle في Gazebo و ROS.	name
.MOVING, WAITING, FALLING	mode
اسم segment الحالي.	current_segment
الموقع على segment.	s
pose في world/Gazebo بعد transform.	x,y,z,yaw

معنى	Field
------	-------

speed سرعة الحركة m/s.

\ ros2 topic echo /room_315/rails/right/shuttles/state

mfja_rail_interfaces/msg/ShuttleState

Shuttle Lifecycle

من لحظة الإضافة حتى الحذف، يمر ال shuttle بهذه المراحل:

ADD WAITING ADD_STOPPED MOVING RESET/OFF REMOVE

الأوامر الأساسية: ADD_STOPPED, ADD_MOVING, ON, OFF, RESET, REMOVE.

Launch Arguments عبر Startup Shuttles

عند تشغيل launch، يمكنك إنشاء shuttles مبدئياً. السلوك صار أبسط الآن: إذا جعلت عدد shuttles أكبر من صفر، فهي

تظهر دائماً في Gazebo على السكة. القرار الوحيد المتبقي هو: هل تبدأ waiting أم تتحرك مباشرة؟

معنى	Argument
عدد shuttles على right rail.	room315_right_shuttle_count
عدد shuttles على left rail.	room315_left_shuttle_count

Argument	معنى
room315_shuttles_start_enabled	إذا false تظهر shuttles وهي waiting. إذا true تظهر وتبدأ moving مباشرة.
room315_right_start_slot	يحدد رقم مكان البداية على right rail: أي slot سيظهر عنده الشاتل، وأي slot سيعود إليه عند RESET.
room315_left_start_slot	نفس الفكرة لكن على left rail.
لا يوجد بعد الآن خيار hidden/deployed عند startup. إذا room315_right_shuttle_count أو room315_left_shuttle_count أكبر من صفر، الشاتل يظهر دائماً. استخدم room315_shuttles_start_enabled فقط لتقرر waiting أو moving.	

الإعدادات	النتيجة عند التشغيل
room315_right_shuttle_count:=1	
room315_right_start_slot:=2	شاتل واحد يظهر على right rail عند slot 2 ويبقى waiting إلى أن ترسل له ON.
room315_shuttles_start_enabled:=false	
room315_right_shuttle_count:=1	
room315_right_start_slot:=4	شاتل واحد يظهر عند slot 4 ويبدأ moving مباشرة.
room315_shuttles_start_en	

الإعدادات	النتيجة عند التشغيل
-----------	---------------------

abled:=true

room315_right_shuttle_cou
nt:=0

لا يتم إنشاء startup shuttle على right rail. يمكنك إضافته لاحقاً runtime باستخدام ADD_STOPPED أو ADD_MOVING.

تخيلها مثل سيارة: start_slot هو رقم الموقف الذي ستقف فيه السيارة، و start_enabled هو هل المحرك شغال من البداية أم السيارة واقفة تنتظر أمر التشغيل.

Runtime Add: ساكن أو متحرك مباشرة

هذه هي feature الأخيرة التي أضفناها. لم نضف field جديد إلى ShuttleCommand.msg. استعملنا حقل command نفسه بطريقة أوضح.

السلوك	Command
ADD_STOPPED ينشئ shuttle ظاهر، لكنه disabled/waiting حتى ترسل ON.	
ADD_MOVING ينشئ shuttle ويبدأ الحركة مباشرة.	
ADD alias قديم، بقي moving مباشرة للتوافق.	

هذه الطريقة أفضل من تغيير message definition لأنها لا تكسر الأوامر القديمة ولا تحتاج migration كبير.

مثال: إضافة Shuttle ساكن ثم تحريكه

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_1', command: 'ADD_STOPPED', start_slot: '2', speed: 0.2}"
```

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_1', command: 'ON'}"
```

ماذا يحدث داخلياً؟

1. ShuttleCommand يستقبل callback.

2. parser يقرأ 'command='ADD_STOPPED

3. enabled=False يقرر

4. ManagedShuttle ينشئ في mode WAITING

5. عند أمر ON يتحول إلى enabled ويبدأ MOVING.

مثال: إضافة Shuttle متحرك مباشرة

```
ros2 topic pub --once /room_315/rails/left/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_left_shuttle_2', command: 'ADD_MOVING', start_slot: '1', speed: 0.25}"
```

داخلياً:

1.parser يقرأ ADD_MOVING.

2.enabled=True يقرر.

3.ينشئ shuttle على start slot.

4.إذا speed أكبر من صفر وshuttle visible/active, يبدأ MOVING mode.

القيمة المقبولة للحركة المباشرة هي ADD_MOVING. إذا أردت shuttle يظهر ساكناً استخدم ADD_STOPPED.

ON/OFF/RESET/REMOVE

هذه الأوامر ترسل على topic التحكم، وليس add topic:

/room_315/rails/right/shuttles/command

/room_315/rails/left/shuttles/command

الوظيفة	Command
يفعل shuttle ويجعله يتحرك إذا speed أكبر من صفر.	ON
يوقف الحركة ويبقي model في مكانه.	OFF
يعيد shuttle إلى start slot.	RESET
يحذف model من Gazebo ويفصله من node.	REMOVE

Switch Control

ال switch يغير routing. في public API نستعمل A1 labels إلى A4 وحالات EXTERIOR أو INTERIOR.

```
ros2 topic pub --once /room_315/rails/right/switches/command \
```

```
mfja_rail_interfaces/msg/SwitchCommand \
```

```
"{switches: [{name: 'A1', state: 'INTERIOR'}]}"
```

ال command ليس بالضرورة يصبح actual فوراً. يوجد delay عبر parameter:

```
room315_switch_motion_delay_s:=0.3
```

actual state ينشر على:

```
/room_315/rails/right/switches/state
```

```
/room_315/rails/left/switches/state
```

Stopper Control

ال stopper هو منطق إيقاف قبل switch. نستخدمه حتى نوقف shuttle, نغير switch, ثم نفتح stopper.

```
ros2 topic pub --once /room_315/rails/right/stoppers/command \
```

```
mfja_rail_interfaces/msg/StopperCommand \
```

```
"{stoppers: [{name: 'A1', state: '1'}]}"
```

معنى	State
------	-------

1, CLOSED, STOP يغلق stopper ويوقف shuttle عنده.

0, OPEN, RELEASE يفتح stopper ويسمح بالمرور.

Sensor Feedback

لدينا نوعان رئيسيان:

• approach sensors: تخبرنا أن shuttle يقترب من switch/stopper.

• position sensors: تخبرنا أن shuttle مر بنقطة detector معينة.

```
ros2 topic echo /room_315/rails/right/sensors/feedback \
```

```
mfja_rail_interfaces/msg/SensorFeedback
```

```
ros2 topic echo /room_315/rails/right/sensors/position_feedback \
```

```
mfja_rail_interfaces/msg/SensorFeedback
```

ملاحظة: approach sensor markers لونها cyan/teal في الكود، لكنها غالباً مخفية خلف stopper الأحمر في

Gazebo لأن الموقع نفسه تقريباً.

Manual Workflow: Sensor ثم Stopper ثم Switch

هذا workflow مهم لفهم منطق التحكم اليدوي:

1. راقب sensors/feedback.

2. عندما يظهر shuttle يقترب من A1 مثلاً، أغلق A1 stopper.

3. غيّر A1 switch.

4. افتح stopper حتى يكمل shuttle.

```
ros2 topic echo /room_315/rails/right/sensors/feedback mfja_rail_interfaces/msg/SensorFeedback
```

```
ros2 topic pub --once /room_315/rails/right/stoppers/command  
mfja_rail_interfaces/msg/StopperCommand "{stoppers: [{name: 'A1', state: '1'}]}"
```

```
ros2 topic pub --once /room_315/rails/right/switches/command mfja_rail_interfaces/msg/SwitchCommand  
"{switches: [{name: 'A1', state: 'INTERIOR'}]}"
```

```
ros2 topic pub --once /room_315/rails/right/stoppers/command  
mfja_rail_interfaces/msg/StopperCommand "{stoppers: [{name: 'A1', state: '0'}]}"
```

Device YAML

أماكن slots/sensors/stoppers ليست hard-coded فقط؛ موجودة في YAML:

```
mfja_robot_control_config/config/room_315_kinematics/rail_devices_right.yaml
```

```
mfja_robot_control_config/config/room_315_kinematics/rail_devices_left.yaml
```

مثال entry:

```
position_sensors:
```

```
- name: DZI1R
```

```
segment: A23
```

s_ratio: 0.35

radius_m: 0.08

لتغيير مكان sensor، عدل segment أو s_ratio ثم شغل device validator. المقصود بالـ validator هنا script

موجود في المشروع اسمه:

mfja_robot_control_config/scripts/room_315_devices_validator.py

وظيفته أن يفحص ملفي rail_devices_right.yaml و rail_devices_left.yaml ويتأكد أن أسماء segments موجودة

وأن s_ratio بين 0.0 و 1.0 وأن fields المطلوبة موجودة.

Device Position Tool

إذا لديك coordinate من Gazebo وتريد أقرب segment و s_ratio:

```
python3 src/mfja_3rd_floor_gz/mfja_robot_control_config/scripts/room_315_device_position_tool.py \
```

```
--side right \
```

```
--x -10.50 \
```

```
--y -3.20 \
```

```
--z 0.85
```

وإذا تريد كتابة النتيجة مباشرة داخل YAML:

```
python3 src/mfja_3rd_floor_gz/mfja_robot_control_config/scripts/room_315_device_position_tool.py \
```

--side right \

--x -10.50 \

--y -3.20 \

--z 0.85 \

--category position_sensors \

--name DZI1R \

--write

category-- مطلوب فقط عندما تريد --write أو عندما تريد توضيح أين يجب أن يبحث tool داخل

category-- ولا -- rail_devices_*.yaml. إذا كنت فقط تريد حساب أقرب segment و s_ratio فلا تحتاج category--

name. ولأن default هو position_sensors, يمكن حذف category position_sensors-- عند تحديث

position sensor مثل DZI1R.

نفس أمر الكتابة يمكن اختصاره هكذا لأن position_sensors هي القيمة الافتراضية:

python3 src/mfja_3rd_floor_gz/mfja_robot_control_config/scripts/room_315_device_position_tool.py \

--side right \

--x -10.50 \

--y -3.20 \

--z 0.85 \

--name DZI1R \

--write

أما إذا كنت تعدل stoppers أو approach_sensors أو slots، فاكتب --category صراحة حتى يعرف أي tool list يعدل.

Gazebo Services

ال node يتعامل مع Gazebo عبر services:

Service	الاستخدام
world/room_315_only//set_pose	تحريك shuttle model
world/room_315_only/create/	spawn model جديد أو marker
world/room_315_only/remove/	حذف shuttle model

```
ros2 service list | grep -E "set_pose|create|remove"
```

في full floor يصبح world name هو mfja_3rd_floor.

الأساسية Topics

Message	Right rail	Purpose
ShuttleCommand	room_315/rails/right/shuttles// add_command	Add shuttle
ShuttleCommand	room_315/rails/right/shuttles/command/	Control shuttle
ShuttleState	room_315/rails/right/shuttles/state/	Shuttle state
SwitchCommand	room_315/rails/right/switches/command/	Switch command
StopperCommand	room_315/rails/right/stoppers/command/	Stopper command
SensorFeedback	room_315/rails/right/sensors/feedback/	Sensors
نفس الأسماء موجودة تحت /room_315/rails/left/ ... لا left rail.		

Custom Messages

ال messages موجودة في mfja_rail_interfaces/msg.

ShuttleCommand.msg

std_msgs/Header header

string name

string command

string start_slot

float64 speed

لم نضف field جديد لـ stopped/moving لأن command يكفي: ADD_STOPPED أو ADD_MOVING.

إذا أردت مستقبلاً إضافة field مثل bool start_enabled، ستحتاج تعديل msg ثم rebuild.

كيف أضفنا الـ Topics؟

إضافة topic في ROS 2 تمر عادة بهذه الخطوات:

1. تعريف message في mfja_rail_interfaces/msg.

2. إضافة message إلى rosidl_generate_interfaces في CMakeLists.txt.

3. إضافة dependency في package.xml.

4. في node: إنشاء publisher أو subscription.

5. كتابة callback إذا كان subscription.

6. تحديث launch/documentation.

في مشروعنا الـ topics موجودة كـ default strings داخل RIGHT_TOPIC_DEFAULTS و

LEFT_TOPIC_DEFAULTS.

أين نعدل بالكود؟ خريطة الملفات

أريد أن أعدل...	الملف
سلوك shuttle أو parser أو topics	room_315_kinematic_shuttle_node.py

الملف	أريد أن أعدل...
mfja_rail_interfaces/msg/*.msg	typed messages الرسائل
room_315_dual_kinematic_shuttles.launch.py	right/left rail nodes تشغيل
mfja_room_315_bringup/launch/ room_315_only.launch.py	Room 315 launch arguments
mfja_3rd_floor_bringup/launch/full_floor.launch.py	Full floor launch arguments
rail_devices_right.yaml, rail_devices_left.yaml	sensors/stoppers/slots مواقع

Topic Defaults في الكود

في أعلى room_315_kinematic_shuttle_node.py يوجد maps: RIGHT_TOPIC_DEFAULTS و

.LEFT_TOPIC_DEFAULTS

```
RIGHT_TOPIC_DEFAULTS = {
```

```
  'add_shuttle_command_topic': '/room_315/rails/right/shuttles/add_command',
```

```
  'switch_command_topic': '/room_315/rails/right/switches/command',
```

```
  'stopper_command_topic': '/room_315/rails/right/stoppers/command',
```

```
  'sensor_state_topic': '/room_315/rails/right/sensors/feedback',
```

```
}
```


إذا أردت تغيير topic name افتراضياً، هذا أحد الأماكن التي تنظر إليها، لكن الأفضل غالباً تمريره ك parameter.

Node Parameters داخل

داخل constructor يتم استدعاء declare_parameter. هذه تعطي node إعدادات قابلة للتمرير من launch.

```
self.declare_parameter('shuttle_count', 1)
```

```
self.declare_parameter('start_enabled', False)
```

```
self.declare_parameter('speed', 0.25)
```

```
self.declare_parameter('enable_gazebo_set_pose', False)
```

عند تعديل behavior يحتاج configuration، أصف parameter بدل hard-code. ثم مرره من launch إذا احتجته.

Subscriptions و Publishers

ال publisher يرسل state. ال subscription يستقبل commands.

```
self.add_shuttle_subscription = self.create_subscription(
```

```
    RailShuttleCommand,
```

```
    add_shuttle_command_topic,
```

```
    self.on_add_shuttle_command,
```

```
    10,
```

)

```
self.state_publisher = self.create_publisher(
```

```
RailShuttleState,
```

```
shuttle_state_topic,
```

```
10,
```

)

الرقم 10 هو QoS depth بسيط للـ queue.

ADD :استقبال Callback

عند وصول message على callback_on_add_shuttle_command، add topic يعمل.

```
def _on_add_shuttle_command(self, message):
```

```
entity_name, slot, speed, enabled = self._parse_add_shuttle_typed_command(message)
```

```
shuttle = self._create_managed_shuttle(
```

```
entity_name=entity_name,
```

```
slot=slot,
```

```
speed=speed,
```

```
enabled=enabled,
```

)

```
self._finish_add_shuttle(shuttle)
```

لاحظ أن أهم نتيجة جديدة هي `.enabled`.

ADD_MOVING و Parser: ADD_STOPPED

أضفنا function تفهم command string وتعيد boolean:

```
def _add_shuttle_start_enabled_from_command(raw_command):
```

```
    if command == 'ADD_MOVING':
```

```
        return True
```

```
    if command == 'ADD_STOPPED':
```

```
        return False
```

```
    return None
```

هذه function تجعل معنى الأمر واضحاً جداً: هناك حالتان فقط عند إضافة shuttle، ساكن أو متحرك مباشرة.

Create Managed Shuttle

function `_create_managed_shuttle` مسؤولة عن object الداخلي. إذا `enabled=False` يبدأ mode `≤`

WAITING ويظهر `stopped_by ≤ DISABLED`.

```
mode = MOVING if enabled and deployed and speed > 0.0 else WAITING
```

كلمة deployed هنا state داخلي معناها أن model موجود حالياً في Gazebo. لم نعد نعرض launch argument
يُسمح بجعل startup shuttles مخفية؛ أي startup shuttle بعدد أكبر من صفر يبدأ deployed=True دائماً.
لذلك ADD_STOPPED لا يحتاج logic خاص في tick؛ يكفي أن يبدأ enabled=False.

Tick Loop

كل فترة زمنية، node يشغل tick_:

1. يحدّث device markers.

2. يطبق switch/stopper pending updates.

3. يمر على كل shuttle.

4. إذا shuttle disabled، يبقى WAITING.

5. إذا enabled، يحسب الحركة مع guards.

6. ينشر state و sensor feedback.

هذه الحلقة هي قلب runtime behavior.

كيف تزداد قيمة s؟ الفكرة الرياضية

قيمة s هي المسافة المقطوعة على current_segment ابتداءً من بداية ذلك segment. يعني إذا كان shuttle على

segment طوله L، فالقيمة الطبيعية لـ s تكون:

$$0 \leq s \leq L$$

كل tick، الـ node يحسب الزمن الذي مر منذ tick السابق. في الكود:

$dt = \max(0.0, (now - self.last_tick).nanoseconds / 1e9)$

رياضياً:

$dt = t_now - t_previous$

حيث dt بالثواني. إذا كان update rate حوالي 30 Hz، فالزمن التقريبي بين ticks:

$dt \approx 1 / 30 = 0.0333333 \text{ s}$

بعد ذلك يستخدم الكود سرعة shuttle، وهي speed أو v بوحدة meter/second. المسافة التي يجب أن يقطعها خلال هذا

tick هي:

$\Delta s = v * dt$

فإذا كانت السرعة $v = 0.2 \text{ m/s}$ و $dt = 0.0333333 \text{ s}$:

$\Delta s = 0.2 * 0.0333333 = 0.0066666 \text{ m}$

يعني shuttle يتقدم تقريباً 6.7 mm في كل tick. إذا لم يوجد stopper أو collision أو نهاية segment، تصبح المعادلة

البسيطة:

$s_new = s_old + v * dt$

هذه هي قاعدة الحركة الأساسية. كل ما حولها في الكود هو حماية: لا نتجاوز نهاية segment، لا نعبّر stopper مغلق، ولا

نقترب من shuttle آخر أكثر من اللازم.

المعادلة داخل KinematicShuttleCore.step

الحركة الفعلية تحدث في الملف:

mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py

داخل function:

```
def step(self, dt, switch_states=None)
```

الكود يحسب أولاً:

```
remaining_distance = self.state.speed * dt
```

رياضياً:

$$R = v * dt$$

حيث R هي المسافة التي ما زال يجب صرفها خلال هذا tick. لماذا لا نسميها مباشرة Δs فقط؟ لأن shuttle قد يكون قريباً من نهاية segment. إذا كانت المسافة المطلوبة أكبر من المتبقي في segment، يجب صرف جزء منها للوصول إلى النهاية، ثم الانتقال إلى segment التالي وصرف الباقي هناك.

```
segment = network.segments[current_segment]
```

```
L = segment.length
```

```
d_end = max(0, L - s)
```

المعنى:

رمز	معنى
L	طول الـ current segment.
s	موقع shuttle الحالي على هذا segment.
d_end	المسافة المتبقية حتى نهاية segment.
R	المسافة التي يجب أن يتحركها shuttle خلال هذا tick.

توجد حالتان:

الحالة 1: الحركة لا تصل إلى نهاية segment

if $R < d_end$:

$$s_new = s_old + R$$

$$R = 0$$

رياضياً:

$$s_{\{k+1\}} = s_k + v * dt$$

وهنا يبقى current_segment كما هو.

الحالة 2: الحركة تصل إلى نهاية segment أو تتجاوزها

$$R = R - d_end$$

$$s = L$$

successor = resolve_successor(current_segment, switch_states)

current_segment = successor

s = 0

يعني نصل إلى نهاية segment، ثم نسأل rail graph: ما هو segment التالي؟ إذا وجدنا successor صالح، ينتقل shuttle إلى بداية successor وتصبح s = 0 هناك. إذا بقي جزء من R، يستمر while loop ويصرف هذا الجزء على segment الجديد.

مثال رقمي: زيادة s خطوة خطوة

لنفرض:

speed v = 0.2 m/s

dt = 0.033333 s

segment length L = 1.000 m

s0 = 0.400 m

المسافة في كل tick:

$\Delta s = v * dt = 0.2 * 0.033333 = 0.006667 \text{ m}$

قيمة s	المعادلة	tick
0.400000	قيمة البداية	0

قيمة s	المعادلة	tick
--------	----------	------

$$0.406667 \quad s_1 = 0.400000 + 0.006667 \quad 1$$

$$0.413334 \quad s_2 = 0.406667 + 0.006667 \quad 2$$

$$0.420001 \quad s_3 = 0.413334 + 0.006667 \quad 3$$

إذا لم يحدث أي توقف، يمكن كتابة العلاقة بعد n ticks:

$$s_n = s_0 + n * v * dt$$

لكن هذه العلاقة صالحة فقط طالما لم نصل إلى نهاية segment. عدد ticks التقريبي للوصول إلى النهاية:

$$n_{to_end} = \text{ceil}((L - s_0) / (v * dt))$$

في المثال:

$$n_{to_end} = \text{ceil}((1.000 - 0.400) / 0.006667)$$

$$= \text{ceil}(90.0)$$

$$= 90 \text{ ticks}$$

والزمن اللازم للوصول إلى النهاية:

$$t_{to_end} = (L - s_0) / v$$

$$= 0.600 / 0.2$$

$$= 3.0 \text{ s}$$

ماذا يحدث عند نهاية segment؟

افترض أن shuttle قريب جداً من نهاية segment:

$$L = 1.000 \text{ m}$$

$$s_{\text{old}} = 0.980 \text{ m}$$

$$v = 0.2 \text{ m/s}$$

$$dt = 0.2 \text{ s}$$

المسافة المطلوبة خلال tick:

$$R = v * dt = 0.2 * 0.2 = 0.040 \text{ m}$$

المسافة المتبقية حتى نهاية segment:

$$d_{\text{end}} = L - s_{\text{old}} = 1.000 - 0.980 = 0.020 \text{ m}$$

بما أن $R \geq d_{\text{end}}$ ، فالحركة تكفي للوصول للنهاية وتبقى مسافة إضافية:

$$R_{\text{after_end}} = R - d_{\text{end}} = 0.040 - 0.020 = 0.020 \text{ m}$$

الكود يضع:

$$s = L$$

$$\text{current_segment} = \text{successor}$$

$$s = 0$$

ثم يستعمل المسافة الباقية 0.020 m على الـ successor:

```
s_on_successor = 0.000 + 0.020 = 0.020 m
```

لذلك عند عبور نهاية segment لا تضيع المسافة الزائدة. الكود يحافظ عليها في remaining_distance ويكمل بها على segment التالي.

كيف يقرر الكود الـ successor؟

عند نهاية shuttle، segment لا يعرف وحده أين يذهب. rail network يحتوي routing table. الكود يستدعي:

```
successor = network.resolve_successor(current_segment, switch_states)
```

إذا كان route ثابتاً، فالـ successor محدد مباشرة. إذا كان segment ينتهي عند switch، يقرأ الكود حالة switch:

```
switch_state = switch_states[switch_name]
```

ثم يختار branch حسب state:

```
EXTERIOR / E -> G branch
```

```
INTERIOR / I -> S branch
```

رياضياً، نستطيع كتابة successor كدالة:

```
next_segment = f(current_segment, switch_states)
```

إذا لم يوجد successor صالح:

```
mode = FALLING
```

وهذا يعني أن graph لم يعط مساراً صالحاً بعد نهاية segment. عندها لا تستمر زيادة s بشكل طبيعي، لأن shuttle خرج من path المعروف.

كيف يتحول s إلى x/y/z/yaw

قيمة s وحدها ليست pose في Gazebo. هي فقط مسافة على segment. لتحويلها إلى position، يستدعي الكود:

```
point, yaw = segment.sample(s)
```

عند استخدام polyline، لدينا نقاط CSV:

P0, P1, P2, ...

يحسب الكود أولاً cumulative arc lengths:

$S_0 = 0$

$S_1 = \text{distance}(P_0, P_1)$

$S_2 = S_1 + \text{distance}(P_1, P_2)$

...

إذا وقع s بين S_i و S_{i+1} ، يحسب:

$$r = (s - S_i) / (S_{i+1} - S_i)$$

ثم interpolation خطي:

$$P(s) = P_i + r * (P_{i+1} - P_i)$$

وبالنسبة للاتجاه:

$$\text{yaw} = \text{atan2}(y_{i+1} - y_i, x_{i+1} - x_i)$$

لذلك S لا يعني x ولا y . هو عدّاد مسافة على المسار، وبعدها يتم تحويله إلى $x/y/z/yaw$.

Cubic Hermite: المعادلات المستخدمة للمسار السلس

ال default backend هو `cubic_hermite`. هنا لا نتحرك بخطوط مستقيمة فقط بين نقاط CSV، بل نستعمل `curve` ناعم

بين كل نقطتين.

لكل `interval` بين P_0 و P_1 يوجد `parameter` اسمه u :

$$0 \leq u \leq 1$$

$u=0$ يعني بداية `interval`، و $u=1$ يعني نهايته. معادلات Hermite basis في الكود:

$$h_{00} = 2u^3 - 3u^2 + 1$$

$$h_{10} = u^3 - 2u^2 + u$$

$$h_{01} = -2u^3 + 3u^2$$

$$h_{11} = u^3 - u^2$$

ومعادلة النقطة:

$$P(u) = h_{00} * P_0 + h_{10} * h * M_0 + h_{01} * P_1 + h_{11} * h * M_1$$

رمز	معنى
-----	------

نقطتا بداية ونهاية interval. P0, P1

tangents عند النقطتين. M0, M1

h طول chord تقريباً بين النقطتين على أساس arc length الأصلي.

نسبة داخل interval من 0 إلى 1. u

لكن انتبه: الحركة عندنا لا تزيد u مباشرة. الحركة تزيد s كمسافة. لذلك يبني الكود جدولاً اسمه arc_map يربط بين:

s -> interval index + u

عند sampling، يبحث عن أقرب قيمتين في arc_map حول s، ثم يقدر u بالاستيفاء:

ratio = (s - lower_s) / (upper_s - lower_s)

u = lower_u + ratio * (upper_u - lower_u)

بعدها يحسب P(u) و tangent. وال yaw يأتي من tangent:

yaw = atan2(tangent_y, tangent_x)

تأثير Stopper على زيادة s

إذا كان stopper مغلقاً أمام shuttle، لا يسمح الكود لـ s أن يتجاوز نقطة التوقف. أولاً يحسب المسافة بين shuttle ونقطة

التوقف:

d_stop = stop_s - s

إذا كانت d_stop موجبة أو صفرية، فالـ stopper أمام shuttle على نفس segment. إذا كان الوقت اللازم للوصول إليه

أقل من tick الحالي:

$$t_stop = d_stop / v$$

ثم:

if $t_stop \leq dt$:

$$effective_dt = t_stop$$

هذا يعني أن الكود لا يحرك shuttle طوال مدة tick، بل فقط حتى وقت الوصول إلى stopper. بعد الحركة يثبت:

$$s = stop_s$$

$$mode = WAITING$$

$$stopped_by = stopper_name$$

مثال:

$$s = 0.900 \text{ m}$$

$$stop_s = 0.950 \text{ m}$$

$$v = 0.2 \text{ m/s}$$

$$dt = 0.5 \text{ s}$$

$$d_{\text{stop}} = 0.950 - 0.900 = 0.050 \text{ m}$$

$$t_{\text{stop}} = 0.050 / 0.2 = 0.25 \text{ s}$$

لأن $t_{\text{stop}}=0.25$ أصغر من $dt=0.5$ ، سيتحرك shuttle فقط نصف tick تقريباً، ويوقف تماماً عند $s=0.950$.

تأثير Collision Avoidance على s

إذا كان هناك shuttle آخر قريب، الكود يجرب الحركة المقترحة أولاً:

```
proposed_pose = core.step(dt)
```

إذا كانت النتيجة آمنة، يعتمد عليها. إذا كانت ستسبب اقتراباً خطيراً، يعمل binary search على الزمن. رياضياً، يبحث عن أكبر

زمن آمن:

$$0 \leq t_{\text{safe}} \leq dt$$

في كل iteration:

$$\text{mid} = (\text{low} + \text{high}) / 2$$

إذا الحركة حتى mid تسبب collision:

$$\text{high} = \text{mid}$$

وإذا كانت آمنة:

$$\text{low} = \text{mid}$$

بعد عدة iterations، يتحرك shuttle بالزمن الآمن low فقط:

$$s_{\text{new}} \approx s_{\text{old}} + v * \text{low}$$

ثم يضع:

mode = WAITING

blocked_by = other_shuttle_name

لذلك collision avoidance لا يغير معادلة الحركة الأساسية، لكنه يبدل dt الكبير بـ زمن أصغر آمن.

Motion Guards

الحركة ليست مجرد زيادة S. قبل الحركة، node يتحقق من:

• هل shuttle enabled ؟

• هل يوجد stopper مغلق أمامه ؟

• هل يوجد shuttle آخر قريب ؟

• هل routing يؤدي إلى successor صالح ؟

إذا فشل route قد يدخل shuttle mode FALLING. إذا كان قريباً من shuttle آخر يدخل WAITING.

Collision Avoidance

الـ node يستخدم distance threshold بين shuttles. القيمة الافتراضية تقريباً:

enable_collision_avoidance = true

shuttle_collision_distance_m = 0.33

إذا shuttle متحرك سيصير قريباً جداً من shuttle آخر، يتوقف عند آخر pose آمن.

هذا collision avoidance منطقي/kinematic وليس Gazebo physics collision.

Remove, Set Pose, و Gazebo Spawn

ثلاث عمليات مهمة:

- request_spawn_if_needed_: يطلب create إذا model غير موجود.

- send_gazebo_pose_: يرسل pose إلى set_pose.

- remove_shuttle_: يحذف model ويزيله من القائمة.

```
ros2 service list | grep /world/room_315_only
```

إذا services غير موجودة، غالباً gazebo world name غير صحيح أو launch لم يبدأ بعد.

Switch State Updates

ال switch command يصل كطلب. ثم node يطبق actual state بعد delay. هذا يسمح بمحاكاة زمن حركة switch.

```
room315_switch_motion_delay_s:=0.3
```

عند تطبيق actual state، ينشر node على:

```
/room_315/rails/right/switches/state
```

ويستطيع أيضاً نشر visual switch command إلى /mfja/conveyor/switch_cmd.

Stopper State Updates

ال stopper مشابه لل switch: command ثم actual state بعد delay.

```
room315_stopper_motion_delay_s:=0.1
```

عند إغلاق stopper، لا يغير rail route، بل يوقف shuttle قبل switch. لذلك هو عنصر safety/control مختلف عن

switch.

Sensor Generation

sensors ليست physical sensors داخل Gazebo. هي virtual sensors يحسبها node من موقع shuttle على

path.

• إذا shuttle داخل radius position sensor، ينشر reading active.

• إذا shuttle يقترب من stopper ضمن distance، ينشر approach feedback.

الرسالة:

```
SensorFeedback
```

```
Header header
```

```
SensorReading[] readings
```

Gazebo في Device Markers

markers تساعدك ترى slots و sensors و stoppers. الكود يعرف ألواناً وأشكالاً داخل DEVICE_MARKER_STYLES.

Shape	Color	Device
-------	-------	--------

sphere green slot

sphere blue position_sensor

sphere cyan/teal approach_sensor

cylinder red stopper

إذا لم ترَ cyan، فهذا طبيعي غالباً لأنه في نفس موقع stopper الأحمر.

Launch Files

ال launch files تجمع كل شيء:

room_315_only.launch.py: يبدأ robots، Room 315 world، اختياريًا، rail nodes اختياريًا.

•full_floor.launch.py: يبدأ full floor world.

room_315_dual_kinematic_shuttles.launch.py: يبدأ right/left shuttle nodes.

```
ros2 launch mfja_room_315_bringup room_315_only.launch.py \
```

```
  robots:=none \
```

```
  start_paused:=false \
```

```
  gui:=true \
```

```
  enable_room315_kinematic_shuttles:=true
```

كيف نصيف Topic جديد؟

1. قرر اسم topic namespace. مثال: `room_315/rails/right/my_feature/command/`.

2. قرر message type. هل تستخدم message موجود أم نصيف جديد؟

3. أضف default topic في `RIGHT_TOPIC_DEFAULTS` و `LEFT_TOPIC_DEFAULTS`.

4. أضف `declare_parameter` إذا تريد `topic configurable`.

5. أنشئ `create_subscription` أو `create_publisher`.

6. اكتب `callback` أو `publish function`.

7. وثق الأوامر في `HTML/README`.

كيف نصيف Message جديد؟

إذا لا يكفي message موجود، أضف message جديد:

1. أنشئ ملف `mfja_rail_interfaces/msg/NewMessage.msg`.

2. أضفه في `rosidl_generate_interfaces`.

3. تأكد من `dependencies` في `package.xml`.

4. استخدمه في `Python import` بعد `build`.

```
colcon build --packages-select mfja_rail_interfaces mfja_robot_control_config
```

```
source install/setup.bash
```

```
ros2 interface show mfja_rail_interfaces/msg/NewMessage
```

تغيير message قد يتطلب rebuild لكل package يعتمد عليه. لذلك إذا feature بسيطة يمكن تمثيلها داخل field موجود، قد يكون ذلك أفضل.

Debugging عملي

المشكلة	أمر فحص
لا أرى topics	ros2 topic list grep room_315
Gazebo لا يتحرك	ros2 service list grep set_pose
ADD مرفوض	افحص start slot occupied في .logs/state
Shuttle لا يتحرك	افحص .enabled, mode, speed
Switch لا يؤثر	أرسل command إلى rail topic وليس visual topic فقط.
Sensor لا يظهر راقب	sensors/feedback أثناء مرور shuttle قرب stopper.

أمثلة تشغيل كاملة

Room 315 مع shuttle ساكن

```
ros2 launch mfja_room_315_bringup room_315_only.launch.py \
```

```
robots:=none \
```

```
start_paused:=false \
```

```
gui:=true \
```

```
enable_room315_kinematic_shuttles:=true
```

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_1', command: 'ADD_STOPPED', start_slot: '2', speed: 0.2}"
```

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_1', command: 'ON'}"
```

Room 315 مع shuttle متحرك مباشرة

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{command: 'ADD_MOVING', start_slot: '3', speed: 0.2}"
```

Checklist نهائية

قبل أن تعتبر feature جديدة جاهزة، راجع هذه القائمة:

- هل يوجد topic واضح و message مناسب؟

- هل الأسماء ضمن namespace صحيح مثل /.../room_315/rails/right/

•هل parser يعطي error مفهوم للأوامر الخاطئة؟

•هل بقيت الأوامر النهائية واضحة وبدون aliases قديمة؟

•هل أضفنا examples في README وHTML؟

•هل جربنا py_compile أو build؟

•هل state topic يوضح النتيجة؟

•هل Gazebo services جاهزة؟

الخلاصة: المشروع مبني على ROS topics typed messages + kinematic core + Gazebo services. عندما

تفهم هذه الطبقات الثلاث، تصبح إضافة features جديدة مثل ADD_STOPPED وADD_MOVING عملية مباشرة

ومنظمة.

مسار المبتدئ المطلق: كيف تقرأ هذا المشروع؟

إذا لم تعمل سابقاً بـ ROS 2، لا تبدأ من الكود مباشرة. ابدأ من السؤال: ما هي البرامج التي تعمل؟ ما هي الرسائل التي تتبادلها؟

أين يدخل أمر المستخدم؟

الخريطة التي سنستخدمها في كل شرح

السؤال	جوابه في مشروعنا	الملف أو الأمر
ما البرنامج الذي يتحكم بال shuttles؟	ode.py	room_315_kinematic_shuttle_node.py
	room_315_kinematic_shuttle_n	mfja_robot_control_config/scripts/

السؤال	جوابه في مشروعنا	الملف أو الأمر
ما شكل أوامر ال- ؟shuttle	ShuttleCommand.msg	mfja_rail_interfaces/msg/ ShuttleCommand.msg
أين نرسل أمر add؟	room_315/rails/right/ shuttles/add_command	ros2 topic pub
أين نقرأ state؟	room_315/rails/right/ shuttles/state	ros2 topic echo
أين نتحدد launch ؟options	room_315_only.launch.py	mfja_room_315_bringup/launch/ room_315_only.launch.py

سنكرر هذا النمط دائماً: مفهوم ROS، ثم أين يظهر في المشروع، ثم الكود، ثم تجربة عملية.

ما هو ROS 2 عملياً؟

ROS 2 ليس robot simulator. هو نظام تواصل وتنظيم للبرامج التي تتحكم بالروبوتات أو simulations. في مشروعنا، Gazebo يعرض العالم، أما ROS 2 فيرسل الأوامر ويقرأ الحالات.

مثال من مشروعنا

عندما تريد إضافة shuttle، لا تفتح function وتستدعيها. بل ترسل message:

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_1', command: 'ADD_STOPPED', start_slot: '2', speed: 0.2}"
```

هذا الأمر يدخل إلى ROS 2، ثم يصل إلى callback داخل الملف:

```
mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
```

ال callback الذي يستقبل الأمر اسمه:

```
_on_add_shuttle_command
```

ما هو Package؟

في ROS 2، ال package هو مجلد منظم يحتوي كود أو messages أو launch files. كل package عنده عادة package.xml، وأحياناً CMakeLists.txt.

Packages في مشروعنا

Package	لماذا موجود؟	مثال ملف
mfja_rail_interfaces	يحتوي message definitions.	msg/ShuttleCommand.msg
mfja_robot_control_config	يحتوي node وال scripts وال	scripts/ room_315_kinematic_shuttle_node.py
mfja_room_315_bring_up	يشغل Room 315 بسهولة.	launch/room_315_only.launch.py
mfja_3rd_floor_description	يحتوي Gazebo models/worlds.	worlds/room_315_only.world

إذا أردت معرفة dependencies لأي package، افتح package.xml.

package.xml: ماذا يخبرنا؟

افتح:

```
mfja_robot_control_config/package.xml
```

ستجد dependencies مثل:

```
<exec_depend>rclpy</exec_depend>
```

```
<exec_depend>mfja_rail_interfaces</exec_depend>
```

```
<exec_depend>ros_gz_interfaces</exec_depend>
```

```
<exec_depend>ros_gz_sim</exec_depend>
```

لماذا نحتاجها؟	Dependency
مكتبة كتابة ROS 2 nodes بلغة Python.	rclpy
mfja_rail_interfaces حتى يستطيع node استخدام messages مثل ShuttleCommand.	
messages/services للتواصل مع Gazebo.	ros_gz_interfaces
تشغيل وتكامل Gazebo مع ROS.	ros_gz_sim

CMakeLists.txt: لماذا يهمنا؟

في package mfja_rail_interfaces, ملف CMakeLists.txt يقول لـ ROS كيف يبني messages.

```
rosidl_generate_interfaces(${PROJECT_NAME}
```

```
"msg/NamedState.msg"
```

```
"msg/SwitchCommand.msg"
```

```
"msg/StopperCommand.msg"
```

```
"msg/ShuttleCommand.msg"
```

```
"msg/ShuttleState.msg"
```

```
"msg/SensorFeedback.msg"
```

```
DEPENDENCIES std_msgs
```

```
)
```

هذا يعني: عندما تعمل ROS، colcon build، يولد Python/C++ types لهذه messages. لذلك نستطيع في Python أن

نكتب:

```
from mfja_rail_interfaces.msg import ShuttleCommand as RailShuttleCommand
```

إذا أضفت ملف msg جديد ولم تضيفه هنا، لن يعرفه ROS.

ما هو Node؟ شرح من الصفر

الـ Node هو برنامج صغير داخل ROS. يمكنه أن يرسل messages، يستقبل messages، يستخدم services، ويملك parameters.

Node في مشروعنا

الملف:

```
mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
```

داخله class تمثل node:

```
class Room315KinematicShuttleNode(Node):
```

هذا يعني أن class يرث من `rclpy.node.Node`. ومن خلال هذا الوراثة يستطيع استعمال:

• `create_subscription`

• `create_publisher`

• `create_client`

• `declare_parameter`

• `get_logger`

كيف يبدأ Node؟

في ROS 2 Python، عادة يوجد function اسمها `main` في آخر الملف. وظيفتها تهيئة ROS وتشغيل `node`. في هذا النوع

من الملفات ستجد نمطاً مشابهاً:

```
rclpy.init()
```

```
node = Room315KinematicShuttleNode()
```

```
rclpy.spin(node)
```

```
rclpy.shutdown()
```

معنى spin: ابقَ شغلاً، واستقبل callbacks، ونفذ timers، ولا تخرج فوراً.

إذا لم يعمل spin، فلن يستقبل node أي topic messages. لذلك ROS program عادة لا ينتهي مباشرة.

ما هو Topic؟ شرح من المشروع

ال Topic هو قناة اسمها string. البرامج ترسل وتستقبل عليها. لا يوجد function call مباشر بين terminal وال node. يوجد message على topic.

Topic الإضافة

```
/room_315/rails/right/shuttles/add_command
```

أين معرف في الكود؟

في الملف:

```
mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
```

داخل map اسمه:

```
RIGHT_TOPIC_DEFAULTS = {  
  
  'add_shuttle_command_topic': '/room_315/rails/right/shuttles/add_command',  
  
}
```

هذا default. يمكن تغييره كـ parameter, لكن غالباً نستخدمه كما هو.

ما هو Subscription؟

الـ Subscription يعني: هذا node مهتم برسائل topic معين. عندما تصل رسالة، شغل callback.

مثال من مشروعنا

```
self.add_shuttle_subscription = self.create_subscription(  
  
  RailShuttleCommand,  
  
  add_shuttle_command_topic,  
  
  self._on_add_shuttle_command,  
  
  10,  
  
)
```

السطر	معناه
RailShuttleCommand	نوع الرسالة المتوقع.
add_shuttle_command_topic	اسم topic.

function self._on_add_shuttle_command التي تعمل عند وصول message.

10 queue depth.

ما هو Callback؟

ال callback هو function يناديه ROS عندما تصل message. أنت لا تستدعيها بيدك.

shuttle إضافة Callback

```
def _on_add_shuttle_command(self, message: RailShuttleCommand) -> None:
```

```
    entity_name, slot, speed, enabled = self._parse_add_shuttle_typed_command(message)
```

```
    shuttle = self._create_managed_shuttle(
```

```
        entity_name=entity_name,
```

```
        slot=slot,
```

```
        speed=speed,
```

```
        enabled=enabled,
```

```
    )
```



```
self._finish_add_shuttle(shuttle)
```

إذا كنت مبتدئاً، اقرأها هكذا:

1.خذ message.

2.حللها إلى name/slot/speed/enabled

3.أنشئ object داخلي لل shuttle.

4.أضفه للنظام واطلب spawn إذا يلزم.

ما هو Publisher؟

ال Publisher يرسل state أو feedback إلى باقي النظام. مثلاً node ينشر shuttle state حتى نستطيع مراقبته.

مثال من الكود

```
self.state_publisher = self.create_publisher(
```

```
RailShuttleState,
```

```
shuttle_state_topic,
```

```
10,
```

```
)
```

ثم لاحقاً، node يبني message وينشره:

```
self.state_publisher.publish(message)
```

الأمر الذي يقرأ ما نشره publisher:

```
ros2 topic echo /room_315/rails/right/shuttles/state \
```

```
mfja_rail_interfaces/msg/ShuttleState
```

ما هو Message؟

ال Message هو schema. يعني شكل البيانات. ROS لا يرسل dictionary عشوائي؛ يرسل نوعاً معروفاً.

ملف message في مشروعنا

```
mfja_rail_interfaces/msg/ShuttleCommand.msg
```

```
std_msgs/Header header
```

```
string name
```

```
string command
```

```
string start_slot
```

```
float64 speed
```

عندما تكتب:

```
"{name: 'room315_right_shuttle_1', command: 'ON'}"
```

ROS يحول النص إلى object من نوع ShuttleCommand.

ما هو Service؟

الـ Service يشبه سؤال وجواب. ترسل request وتنتظر response. نستخدمه مع Gazebo لأننا نطلب عمليات محددة:

.create, remove, set_pose

مثال من الكود

```
self.spawn_client = self.create_client(SpawnEntity, gazebo_spawn_service)
```

```
self.set_pose_client = self.create_client(SetEntityPose, gazebo_set_pose_service)
```

```
self.delete_client = self.create_client>DeleteEntity, gazebo_delete_service)
```

أين تأتي أسماء services؟ من world name:

```
/world/room_315_only/create
```

```
/world/room_315_only/set_pose
```

```
/world/room_315_only/remove
```

افحصها:

```
ros2 service list | grep /world/room_315_only
```

ما هو Parameter؟

الـ Parameter هو قيمة إعداد داخل node. مثلاً: السرعة، عدد shuttles، هل نستخدم Gazebo spawn.

تعريف parameter في الكود

```
self.declare_parameter('speed', 0.25)
```

```
self.declare_parameter('shuttle_count', 1)
```

```
self.declare_parameter('start_enabled', False)
```

قراءة parameter

```
speed = float(self.get_parameter('speed').value)
```

```
start_enabled = bool(self.get_parameter('start_enabled').value)
```

عند launch يمكن تمريرها:

```
ros2 launch ... room315_shuttle_speed:=0.2
```

ما هو Launch File؟

ال launch file يشغل مجموعة nodes ويعطيها parameters. بدل أن تبدأ كل node يدوياً، نكتب launch ينسق التشغيل.

مثال ملف

```
mfja_room_315_bringup/launch/room_315_only.launch.py
```

فيه arguments مثل:

```
DeclareLaunchArgument(
```

```
'room315_shuttles_start_enabled',
```

```
default_value='false',
```

```
choices=['true', 'false'],
```

```
)
```

وتشغله هكذا:

```
ros2 launch mfja_room_315_bringup room_315_only.launch.py \
```

```
room315_shuttles_start_enabled:=true
```

من Launch Argument إلى Parameter

المبتدئ غالباً يخلط بين Launch Argument و Node Parameter. ال launch argument قيمة تدخل لل launch.

بعدها launch يمررها ك parameter إلى node.

في launch

```
'start_enabled': LaunchConfiguration('room315_shuttles_start_enabled')
```

في node

```
start_enabled = bool(self.get_parameter('start_enabled').value)
```

إذا عرّفت argument ولم تمرره في parameters، فلن يؤثر على node.

ما هو Timer أو Tick Loop؟

node لا ينتظر commands فقط. يحتاج أيضاً تحديث حركة shuttles كل فترة. لذلك يستخدم timer يشغل function اسمها غالباً tick_.

في tick_ يحدث الآتي:

1. حساب الزمن منذ آخر tick_.

2. تحديث markers.

3. تطبيق switch/stopper updates.

4. تحريك كل shuttle أو إبقاؤه waiting.

5. نشر state و sensor feedback.

هذا هو "نبض" النظام.

كيف ترتبط ROS مع Gazebo؟

Gazebo هو ROS node simulator. لا يحرك model مباشرة داخل memory، بل يستخدم services مقدمة عبر

ros_gz.

كيف يطلب من Gazebo؟

ماذا يريد ROS؟

إنشاء shuttle model world/room_315_only/create/

world/room_315_only//

تحريك shuttle

set_pose

حذف shuttle /world/room_315_only/remove/

لذلك إذا Gazebo لم يبدأ، أو world name خطأ، ستفشل هذه services.

كيف نقرأ الكود بدون ضياع؟

ملف room_315_kinematic_shuttle_node.py طويل. لا تقرأه من السطر الأول إلى الأخير. اقرأه حسب feature.

إذا تريد فهم إضافة shuttle

1. ابحث عن add_shuttle_command_topic.

2. ابحث عن create_subscription الخاص به.

3. اذهب إلى on_add_shuttle_command_.

4. اذهب إلى parser.

5. اذهب إلى create/finish/spawn.

إذا تريد فهم switch

1. ابحث عن switch_command_topic.

2. اذهب إلى switch callback.

3. ابحث عن pending switch updates.

4. راقب publish switch state.

مثال كامل: Topic جديد من الفكرة إلى التطبيق

لنفترض أنك تريد topic جديد يخبرك بعدد shuttles. سنطبقها كفكرة تعليمية.

1. اختر topic name

```
/room_315/rails/right/shuttles/count
```

2. اختر message type

لا نحتاج custom message. نستخدم:

```
std_msgs/msg/Int32
```

3. أضيف publisher في node

```
from std_msgs.msg import Int32
```

```
self.shuttle_count_publisher = self.create_publisher(
```

```
    Int32,
```

```
    f'/room_315/rails/{self.rail_side}/shuttles/count',
```

```
    10,
```

```
)
```


4. انشر العدد

```
message = Int32()
```

```
message.data = len(self.shuttles)
```

```
self.shuttle_count_publisher.publish(message)
```

هذا المثال يعلمك إضافة publisher بدون custom message.

مثال كامل: Command جديد داخل Topic موجود

هذا ما فعلناه في ADD_STOPPED و ADD_MOVING.

الفكرة

بدل topic جديد، أضف قيمة جديدة في command.

الملف

```
mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
```

المكان

```
_add_shuttle_start_enabled_from_command
```

الكود

```
if command == 'ADD_STOPPED':
```

```
return False
```

```
if command == 'ADD_MOVING':
```

```
    return True
```

هذه الطريقة مناسبة عندما command الجديد نفس message ونفس topic ونفس callback تقريباً.

مثال كامل: Message جديد من الصفر

إذا احتجت data جديدة لا تناسب messages الحالية:

1. أنشئ ملف

```
mfja_rail_interfaces/msg/ShuttleCount.msg
```

2. اكتب fields

```
std_msgs/Header header
```

```
string rail_side
```

```
int32 count
```

3. أضفه في CMakeLists

```
rosidl_generate_interfaces(${PROJECT_NAME}
```

```
"msg/ShuttleCount.msg"
```

```
...
```

```
)
```

Build .4

```
colcon build --packages-select mfja_rail_interfaces mfja_robot_control_config
```

```
source install/setup.bash
```

5. استخدمه في Python

```
from mfja_rail_interfaces.msg import ShuttleCount
```

كيف تختار بين Topic وService؟

سؤال مهم جداً للمبتدئ: هل أضيف topic أم service؟

استخدم **Service** عندما...

استخدم **Topic** عندما...

تريد بث state بشكل مستمر. تريد طلباً واضحاً ورداً واضحاً.

الأمر لا يحتاج response مباشر. تحتاج تعرف هل العملية نجحت فوراً.

عدة nodes قد تستمع لنفس البيانات. الطلب موجه لمزود خدمة محدد.

في مشروعنا: shuttle commands هي topics. Gazebo create/remove هي services.

كيف تفهم Namespaces؟

ال namespace يمنع الفوضى. لدينا right rail و left rail. لو سمينا كل topics /shuttle/command لن نعرف أي rail نقصد.

لذلك نستخدم:

```
/room_315/rails/right/shuttles/command
```

```
/room_315/rails/left/shuttles/command
```

في node، launch، يستخدم namespace:

```
namespace='room_315/rails/right'
```

وهذا يجعل نفس node code يخدم الجهتين إذا تغيرت parameters وال namespace.

كيف تفهم QoS بشكل مبسط؟

في create_publisher(..., 10) أو create_subscription(..., 10) الرقم 10 هو queue depth. يعني ROS يحتفظ بعدد محدود من الرسائل إذا لم تعالج فوراً.

في مشروعنا، commands ليست عالية التردد، لذلك 10 depth مناسب.

كمبتدئ، لا تبدأ بتعديل QoS إلا إذا لديك مشكلة واضحة في ضياع رسائل أو timing.

كيف تفهم Headers؟

كثير من messages تبدأ بـ std_msgs/Header header. هذا يعطي timestamp و frame_id أحياناً.

مثال:

```
std_msgs/Header header
```

```
string name
```

```
string command
```

في commands البسيطة قد لا تهتم بالheader. لكن في states و timestamp، sensor feedback مهم لفهم متى

حدثت القراءة.

كيف تفهم frame_id و world؟

في robotics، ال pose دائماً مرتبط بإطار مرجعي. في مشروعنا غالباً نستخدم world.

:parameter

```
self.declare_parameter('frame_id', 'world')
```

عندما ننشر pose أو state، نفهم أن x,y,z,yaw هي بالنسبة إلى Gazebo world.

كيف تعمل Pose Transform؟

CSV rail data قد يكون في coordinate system مختلف عن Gazebo. لذلك node يحتوي transform

:parameters

pose_transform_a

pose_transform_b

pose_transform_tx

pose_transform_c

pose_transform_d

pose_transform_ty

هذه تحول raw rail coordinates إلى Gazebo coordinates. لذلك عندما ترى sensor في YAML، موقعه النهائي في Gazebo قد يمر عبر transform.

إذا أردت تعديل calibration مؤقتاً، استخدم pose offset topic بدلاً من تغيير CSV مباشرة.

كيف تفهم الـ YAML Configs؟

YAML ملفات إعداد يقرأها الكود عند التشغيل. ليست Python code، لكنها تحدد data.

ماذا يحدد؟	YAML
الـ graph والـ segments والـ switches.	rail_network_right.yaml
rail_devices_right.yaml	slots/sensors/stoppers على rail right.
robots spawn داخل Room 315.	robots_room_315_only.yaml
إذا غيرت YAML، قد لا تحتاج rebuild، لكن غالباً تحتاج relaunch.	

كيف تقرأ robots_room_315_only.yaml؟

الملف المفتوح عندك:

```
mfja_robot_control_config/config/robots_room_315_only.yaml
```

كل robot entry يحتوي:

```
- name: kuka1
```

```
model: kuka_kr6r900sixx
```

```
x_pose: -10.6366
```

```
y_pose: -6.0
```

```
z_pose: 1.0
```

```
yaw: 3.14
```

```
enabled: true
```

ال launch يقرأ هذه القائمة ويقرر أي robots ي spawn حسب robots argument:

كيف يعمل robots:=kuka؟

في launch/control code توجد shortcuts. عندما تكتب:

```
robots:=kuka
```

الكود يطابقها مع robot model يبدأ ب_kuka أو name مناسب. لذلك لا تحتاج كتابة kuka1 دائماً.

أمثلة:

```
robots:=none
```

```
robots:=all
```

```
robots:=kuka
```

```
robots:=kuka,tiago
```

```
robots:=1,5
```

كيف تقرأ State بدل التخمين؟

لا تعتمد على النظر إلى Gazebo فقط. ROS يعطيك state مباشر.

```
ros2 topic echo /room_315/rails/right/shuttles/state \
```

```
mfja_rail_interfaces/msg/ShuttleState
```

إذا shuttle لا يتحرك، انظر إلى:

mode•

speed•

current_segment•

s•

هذه المعلومات أهم من الانطباع البصري.

كيف تعرف أن Command وصل؟

ثلاث طرق:

1. راقب terminal الذي يشغل launch، ستظهر logs.

2. راقب state topic قبل وبعد الأمر.

3. استخدم ros2 topic echo على command topic إذا أردت رؤية الرسائل المنشورة.

```
ros2 topic echo /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand
```

افتح هذا في terminal، ثم من terminal آخر أرسل command. سترى الرسالة كما وصلت.

كيف تستخدم ros2 topic pub بشكل صحيح؟

أمر ros2 topic pub هو الطريقة اليدوية لإرسال رسالة على topic. إذا كنت جديداً على ROS 2، اقرأ الأمر كجملته كاملة:

"انشر message من نوع محدد على topic محدد، وضع داخلها هذه القيم".

البنية العامة:

```
ros2 topic pub --once TOPIC MESSAGE_TYPE "YAML_MESSAGE"
```

مثال تحكم بـ shuttle موجود:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

mfja_rail_interfaces/msg/ShuttleCommand \

"{name: 'room315_right_shuttle_1', command: 'ON'}"

معنى	جزء الأمر
برنامج command line الخاص بـ ROS 2.	ros2
مجموعة أوامر التعامل مع topics.	topic
اختصار publish: أرسل message.	pub
انشر الرسالة مرة واحدة.	once--
القناة التي ترسل عليها. يجب أن تكون node مشتركة في نفس topic حتى تستقبل الرسالة.	TOPIC
نوع الرسالة بصيغة package/msg/MessageName.	MESSAGE_TYPE
القيم التي تملأ fields داخل الرسالة.	YAML_MESSAGE
هذا ليس جزءاً من ROS. في Bash يعني: "الأمر لم ينته، كمل على السطر التالي".	\

أهم نقطة للمبتدئ: TOPIC يحدد "أين تذهب الرسالة"، أما MESSAGE_TYPE يحدد "شكل الرسالة"، وأما

YAML_MESSAGE فيحدد "المحتوى الفعلي للرسالة".

تشرح أمر Switch سطر بسطر

خذ هذا الأمر كمثال:

```
ros2 topic pub --once /room_315/rails/right/switches/command \
```

```
mfja_rail_interfaces/msg/SwitchCommand \
```

```
"{switches: [{name: 'A1', state: 'INTERIOR'}]}"
```

القسم	ماذا يعني؟	على ماذا يدل في مشروعنا؟
room_315/	namespace عام للغرفة.	الأمر يخص Room 315 وليس نظاماً آخر.
rails/	قسم السكك.	نحن لا نرسل أمر robot arm، بل أمر rail system.
right/	اختيار السكة اليمينية.	لو أردت اليسار تستخدم /left.
switches/	الجهاز المستهدف هو switches.	ليس shuttles وليس stoppers.
command/	هذا topic أوامر. نكتب عليه لتغيير الحالة، ولا نقرأ منه state النهائي.	
mfja_rail_interfaces/msg/SwitchCommand	نوع الرسالة.	ROS سيبني object من نوع SwitchCommand ويملاً fields من النص.
switches	field داخل SwitchCommand.	لأنه يمكن إرسال أكثر من switch في نفس الرسالة.
'name: 'A1'	اسم ال switch المطلوب.	نختار switch المحطة A1.
'state: 'INTERIOR'	الحالة المطلوبة.	وجه المسار إلى branch الداخلي.

النتيجة المنطقية هي: A1 على rail right يصبح باتجاه INTERIOR. في الكود هذه القيمة تتحول داخلياً إلى الرمز المختصر

S، بينما EXTERIOR تتحول إلى G.

EXTERIOR أو E -> G

INTERIOR أو I -> S

رحلة أمر Switch داخل الكود

حتى لا يبقى الأمر مجرد نص في terminal، هذه رحلته داخل المشروع:

1. أنت تنشر message على /room_315/rails/right/switches/command/

2. ROS 2.2 يتأكد أن نوع الرسالة هو SwitchCommand.

3. الـ node المشتركة بهذا topic تستقبل الرسالة.

4. callback اسمها on_switch_command تقرأ message.switches.

5. الكود يحول INTERIOR إلى state داخلي مختصر.

6. الكود يحدّث switch state ويؤثر على routing عند وصول shuttle إلى ذلك switch.

مكان الاشتراك بالـ topic

```
self.switch_subscription = self.create_subscription(
```

```
    SwitchCommand,
```

```
    switch_command_topic,
```

```
    self._on_switch_command,
```

10,

)

مكان استقبال الرسالة

```
def _on_switch_command(self, message: SwitchCommand) -> None:

    updates = self._switch_updates_from_named_states(message.switches)

    self._handle_switch_updates(updates, source='typed command')
```

مكان فهم state

```
if state in {'G', 'E', 'EXTERIOR'}:

    return 'G'

if state in {'S', 'I', 'INTERIOR'}:

    return 'S'
```

لذلك عندما تكتب 'state: 'INTERIOR' في terminal، الكود لا يحرك model مباشرة في نفس السطر. هو يحول الأمر إلى state منطقي لل switch، ثم يستخدم هذا state لاحقاً عندما يقرر المسار التالي لل shuttle.

كيف تستخدم ros2 topic echo؟

البنية:

ros2 topic echo TOPIC MESSAGE_TYPE

مثال:

```
ros2 topic echo /room_315/rails/right/sensors/feedback \
```

```
mfja_rail_interfaces/msg/SensorFeedback
```

إذا لا تظهر رسائل، قد يكون السبب أن الحدث لم يحدث بعد. مثلاً `sensor feedback` يظهر عندما يقترب `shuttle` من `.sensor`.

كيف تستخدم `ros2 interface show`؟

هذا الأمر أساسي للمبتدئ لأنه يخبرك ما `fields` المطلوبة داخل `message`. عندما ترى نوعاً مثل

`mfja_rail_interfaces/msg/SwitchCommand`، لا تحفظه بضم. اسأل ROS عنه:

```
ros2 interface show mfja_rail_interfaces/msg/SwitchCommand
```

سيظهر أن `SwitchCommand` يحتوي:

```
std_msgs/Header header
```

```
NamedState[] switches
```

هذا يعني أن الرسالة فيها `field` اسمه `header` و `field` اسمه `switches`. الرمز `[]` يعني `list/array`. نوع عناصر هذه `list` هو

`NamedState`. لذلك نكمل ونفحص `NamedState`:

```
ros2 interface show mfja_rail_interfaces/msg/NamedState
```

سيظهر:

string name

string state

فتعرف أن الرسالة يجب أن تكون مثل:

```
"{switches: [{name: 'A1', state: 'INTERIOR'}]}"
```

لم نكتب header داخل الأمر لأننا لا نحتاجه هنا. ROS 2 يعطيه default value إذا لم نمرره. المهم لهذا command

هو field switches.

من أين جاء mfja_rail_interfaces/msg/SwitchCommand؟

هذه العبارة ليست path عادي على القرص، بل ROS 2 interface type. صيغتها العامة:

package_name/msg/MessageName

القسم	معنى	في مثالنا
package_name	اسم package التي تملك تعريف الرسالة.	mfja_rail_interfaces
msg	نوع message: interface أو service أو action.	msg
MessageName	اسم الرسالة بدون extension.	SwitchCommand

الملف الحقيقي في المشروع هو:

mfja_rail_interfaces/msg/SwitchCommand.msg

لكن عندما نستخدمه في أوامر ROS 2، نحذف msg. وتكتبه بصيغة:

mfja_rail_interfaces/msg/SwitchCommand

نفس القاعدة تنطبق على بقية الرسائل:

ملف في المشروع	النوع الذي تكتبه في terminal	متى نستخدمه؟
mfja_rail_interfaces/msg/ SwitchCommand.msg	mfja_rail_interfaces/msg/ SwitchCommand	إرسال أوامر .switches
mfja_rail_interfaces/msg/ StopperCommand.msg	mfja_rail_interfaces/msg/ StopperCommand	إرسال أوامر .stoppers
mfja_rail_interfaces/msg/ ShuttleCommand.msg	mfja_rail_interfaces/msg/ ShuttleCommand	إضافة أو التحكم بـ .shuttles
mfja_rail_interfaces/msg/ ShuttleState.msg	mfja_rail_interfaces/msg/ ShuttleState	قراءة حالة .shuttles

كيف يتحول ملف .msg إلى نوع يستعمله ROS؟

ملف .msg وحده ليس كافياً. يجب أن يكون مسجلاً في mfja_rail_interfaces/CMakeLists.txt داخل

.rosidl_generate_interfaces

```
rosidl_generate_interfaces(${PROJECT_NAME}
```

```
"msg/NamedState.msg"
```

```
"msg/SwitchCommand.msg"
```


"msg/SwitchState.msg"

"msg/StopperCommand.msg"

"msg/StopperState.msg"

"msg/ShuttleCommand.msg"

"msg/ShuttleState.msg"

"msg/SensorReading.msg"

"msg/SensorFeedback.msg"

DEPENDENCIES std_msgs

)

عند تنفيذ ROS 2، colcon build يولد كوداً داخلياً لهذه messages. لذلك يصبح بإمكان Python node أن يكتب:

```
from mfja_rail_interfaces.msg import SwitchCommand
```

```
from mfja_rail_interfaces.msg import NamedState
```

وبعدها ينشئ subscription يتوقع رسائل من هذا النوع:

```
self.switch_subscription = self.create_subscription(
```

```
    SwitchCommand,
```

```
    switch_command_topic,
```

```
self._on_switch_command,
```

```
10,
```

```
)
```

معنى هذا الكود: "استقبل messages من نوع SwitchCommand على switch_command_topic، وعند وصول رسالة شغل function اسمها _on_switch_command".

كيف تعرف نوع أي Topic بنفسك؟

لا تحتاج أن تخمن ROS 2 message type. يستطيع أن يخبرك.

1. اعرض topics الخاصة بالمشروع

```
ros2 topic list | grep /room_315/rails
```

2. اسأل عن topic محدد

```
ros2 topic info /room_315/rails/right/switches/command
```

سترى شيئاً مثل:

Type: mfja_rail_interfaces/msg/SwitchCommand

Publisher count: ...

Subscription count: ...

الآن عرفت النوع. بعدها افحص fields:

القاعدة العملية: topic info يعطيك نوع الرسالة، و interface show يعطيك شكل الرسالة من الداخل.

كيف تتحول YAML Message إلى Fields؟

آخر جزء من ros2 topic pub هو نص بصيغة قريبة من ROS 2 YAML. يأخذ هذا النص ويحاول تعبئة fields الموجودة

في message type.

```
"{switches: [{name: 'A1', state: 'INTERIOR'}]}"
```

التحويل يحدث بهذا الشكل:

النوع	إلى message field	من النص
array/list	SwitchCommand.switches	switches
NamedState	عنصر داخل switches	<pre>name: 'A1', state: } {"INTERIOR</pre>
string	NamedState.name	name
string	NamedState.state	state

إذا كتبت field غير موجود، أو كتبت شكل list بطريقة خاطئة، سيفشل الأمر قبل أن يصل إلى node. لذلك نستخدم ros2

interface show دائماً قبل كتابة message جديدة.

نفس الرسالة بشكل أوضح

```
{
```

```
switches: [  
  
  {  
  
    name: 'A1',  
  
    state: 'INTERIOR'  
  
  }  
  
]  
  
}
```

شرح ملفات msg. المفتوحة عندك

هذه الملفات هي contract بين ال terminal وال node. أي طرف يريد إرسال أو استقبال message يجب أن يتفق على نفس الشكل.

الملف	المحتوى	المعنى العملي
NamedState.msg	string name	زوج بسيط: اسم device وحالته. مثل A1 / INTERIOR أو
	string state	.A1 / 1
SwitchCommand.msg	std_msgs/Header	
	header	أمر يرسله المستخدم إلى switches. field switches لأنك
	NamedState[]	قد تغير عدة switches معاً.
SwitchState.msg	switches	
	std_msgs/Header	حالة switches المنشورة من node للقراءة. نفس شكل

المعنى العملي	المحتوى	الملف
	header	
command تقريباً، لكن الاتجاه بالعكس: node ينشر وأنت تقرأ.	NamedState[]	
	switches	
	std_msgs/Header	
أمر فتح/إغلاق stoppers. مثال: A1=1 يعني أغلق/stop، و	header	StopperCommand.msg
A1=0 يعني افتح/release.	NamedState[]	
	stoppers	
	std_msgs/Header	
حالة stoppers الحالية التي تنشرها node.	header	StopperState.msg
	NamedState[]	
	stoppers	

الفرق بين Command و State: الـ Command ترسله أنت إلى node لتطلب تغييراً. الـ State تنشره node لتخبرك

بما حدث أو ما هي الحالة الحالية.

نفس NamedState، استخدامات مختلفة

Switch command

```
{switches: [{name: 'A1', state: 'INTERIOR'}]}
```

Stopper command

```
{stoppers: [{name: 'A1', state: '1'}]}
```

لاحظ أن NamedState نفسه بسيط جداً، لكن اسم الـ field الخارجي يغير معنى الرسالة: switches يعني نتحكم بالـ

switches, و stoppers يعني نتحكم بالـ stoppers.

لماذا SwitchCommand يحتوي Array؟

لأنه أحياناً تريد تغيير أكثر من switch في رسالة واحدة.

```
ros2 topic pub --once /room_315/rails/right/switches/command \
```

```
mfja_rail_interfaces/msg/SwitchCommand \
```

```
"{switches: [
```

```
{name: 'A1', state: 'INTERIOR'},
```

```
{name: 'A2', state: 'EXTERIOR'}
```

```
]}"
```

هذا أفضل من إرسال رسالتين إذا أردت update منطقي واحد.

كيف تفهم arrays داخل messages؟

في msg، الرمز [] يعني list/array.

```
NamedState[] switches
```

لذلك في YAML message نكتب:

```
{switches: [{name: 'A1', state: 'INTERIOR'}]}
```

إذا يوجد أكثر من عنصر:

```
{switches: [  
  
  {name: 'A1', state: 'INTERIOR'},  
  
  {name: 'A2', state: 'EXTERIOR'}  
]}
```

كيف تفهم code naming؟

أسماء functions ليست عشوائية:

معنى	Pattern
Callback يستقبل event/message.	..._on_
تحويل نص message إلى values منظمة.	..._parse_
حل قيمة ناقصة أو تحقق من صلاحيتها.	..._resolve_
نشر message على topic.	..._publish_
إرسال request إلى service أو future.	..._request_

عندما تبحث في الكود، هذه patterns تساعدك تعرف وظيفة function قبل قراءة تفاصيلها.

الخلاصة التعليمية للمبتدئ

بعد قراءة هذا المسار يجب أن تكون قادراً على شرح الآتي:

- ROS 2 ينظم التواصل بين programs.

- Node يستقبل وينشر messages.

- Topic هو قناة، و Message هو شكل البيانات.

- Callback هو function تعمل عندما تصل message.

- Launch يشغل nodes ويمرر parameters.

- Room 315 shuttles تتحرك kinematically وليس بعجلات فيزيائية.

- ADD_STOPPED و ADD_MOVING مجرد قيم command جديدة على add topic.

- أي feature جديدة يجب أن تتبع مسار: publish/debug -> state change -> parser -> command.

إذا نسيت كل التفاصيل، ارجع لهذه الجملة: في ROS 2 نحن لا ننادي functions مباشرة، بل ننشر messages على

topics، وال nodes تستقبلها عبر callbacks وتغير state أو تطلب services من Gazebo.

منهج التعلم العملي داخل هذا المشروع

من هذه الصفحة فصاعداً، سنمشي بطريقة أبطأ وأكثر تفصيلاً: ليس فقط ماذا نفعل، بل لماذا نفعل ذلك، وكيف تربط مفاهيم ROS

2 مباشرة مع ملفات مشروع Room 315.

الفكرة التي يجب أن تثبت في ذهنك: كل feature في هذا المشروع تمر غالباً عبر أربع طبقات: Message ثم Topic ثم

Node callback ثم Gazebo action/state publication.

طريقة الدراسة المقترحة

1. افتح التقرير في المتصفح.
2. افتح الكود بجانبه في VS Code.
3. شغل Room 315 في Terminal 1.
4. نفذ أوامر ros2 topic في Terminal 2.
5. كلما قرأت function في التقرير، ابحث عنها في الملف بالاسم.
6. بعد الفهم، طبق تعديل صغير بنفسك ثم اختبره.

ROS 2 من الصفر: لماذا نحتاجه هنا؟

تخيل أننا نريد التحكم بنظام فيه shuttles, rails, switches, stoppers, sensors. بدون ROS سنكتب برنامجاً واحداً ضخماً فيه كل شيء: state, logging, commands, control, simulation. هذا يصعب تطويره وفهمه. ROS 2 يحل هذه المشكلة بفصل النظام إلى أجزاء صغيرة اسمها Nodes. كل Node مسؤول عن وظيفة معينة ويتواصل مع باقي النظام عبر Topics و Services.

مع ROS 2	لو لم نستخدم ROS
عدة Nodes واضحة المسؤولية	كل شيء داخل برنامج واحد
تغيير صغير قد يكسر أجزاء كثيرة	تغيير صغير قد يكسر أجزاء كثيرة
callback أو topic محدد	callback أو topic محدد
صعب مراقبة state	صعب مراقبة state
ros2 topic echo يعرض البيانات مباشرة	ros2 topic echo يعرض البيانات مباشرة
command يرسل ros2 topic pub	command يرسل ros2 topic pub
صعب اختبار أمر واحد	صعب اختبار أمر واحد

لذلك ROS هنا ليس شيئاً إضافياً فقط؛ هو طريقة تنظيم المشروع والتحكم به.

أول Mental Model: Publisher و Subscriber

أهم فكرة في ROS هي أن Publisher لا يعرف من يستقبل الرسالة، و Subscriber لا يعرف من أرسلها. كلاهما يتفقان على اسم Topic ونوع Message.

Publisher ros2 topic pub Topic /room_315/rails/right /shuttles/add_command Subscriber
callback

في مشروعنا، عندما نرسل ADD_STOPPED على add topic، نحن publisher مؤقت. ال node
room_315_kinematic_shuttle_node.py هو subscriber.

كيف تقرأ Topic Name؟

أسماء topics في مشروعنا ليست عشوائية. هي namespace منظم يعطيك معنى من الاسم وحده.

/room_315/rails/right/shuttles/add_command

الجزء	معنى
room_315/	النظام أو المكان.
rails/	Subsystem الخاص بالسكة.

الجزء	معنى
-------	------

right/ أي جهة من السكة: left أو right.

shuttles/ نوع العنصر الذي نتحكم به.

add_command/ نوع العملية: إضافة shuttle.

هذه الطريقة تجعل التوسع أسهل. إذا أردنا إضافة subsystem جديد، نتبع نفس النمط.

كيف تعرف نوع Message لأي Topic؟

قبل أن ترسل command، يجب أن تعرف نوع message. استخدم:

```
ros2 topic info /room_315/rails/right/shuttles/add_command
```

بعدها اعرض شكل الرسالة:

```
ros2 interface show mfja_rail_interfaces/msg/ShuttleCommand
```

سيظهر:

```
std_msgs/Header header
```

```
string name
```

```
string command
```

```
string start_slot
```

```
float64 speed
```

معنى ذلك أن الأمر الذي ترسله يجب أن يحتوي fields مطابقة. ليس ضرورياً ملء كل fields، لكن يجب أن يكون syntax صحيحاً.

شرح ShuttleCommand بالتفصيل

Field	متى يستخدم؟	مثال
name	اسم shuttle. مهم عند التحكم أو إضافة اسم محدد.	room315_right_shuttle_1
command	نوع العملية.	ADD_STOPPED, ON
start_slot	مكان البداية عند add/reset-like creation.	2
speed	سرعة shuttle. إذا صفر يستخدم default.	0.2
	speed.	

نفس message يستخدم في موضوعين مختلفين:

•shuttles/add_command: لإضافة shuttle جديد.

•shuttles/command: للتحكم بـ shuttle موجود.

الفرق ليس في message type، بل في topic الذي ترسل إليه، والcallback الذي يستقبل الرسالة.

لماذا لم نضف Message جديد لـ ADD_STOPPED؟

عندما طلبنا feature "إضافة shuttle ساكن أو متحرك مباشرة"، كان أمامنا خياران:

1. نضيف field جديد مثل start_enabled إلى ShuttleCommand.msg.

2.نستخدم field command ونضيف values جديدة مثل ADD_STOPPED.

اخترنا الخيار الثاني لأنه أبسط وأوضح: لا نحتاج message جديد، ولا نحتاج topic جديد، فقط نحدد قيمتين نهائيتين داخل

ADD_STOPPED command: وADD_MOVING.

الخيار	الميزة	المشكلة
Field جديد	واضح جداً يحتاج rebuild ويمكن يؤثر على مستخدمين آخرين	
Command values جديدة توافق خلفي وسهل		يجب توثيق القيم جيداً

من Terminal إلى Function: رحلة ADD_STOPPED

لنمشي خطوة بخطوة:

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_1', command: 'ADD_STOPPED', start_slot: '2', speed: 0.2}"
```

1. ros2 topic pub ييني message من النص.

2. ROS يرسل message على topic.

3. Subscription داخل node يستقبلها.

4. callback_on_add_shuttle_command يعمل.

5. parser_parse_add_shuttle_typed_command يقرأ fields.

6. add_shuttle_start_enabled_from_command يحول ADD_STOPPED إلى False.

7. create_managed_shuttle ينشئ shuttle داخلياً.

8. finish_add_shuttle يضيفه للقائمة ويطلب spawn إذا احتاج.

تتبع الكود بيدك

افتح الملف:

mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py

ابحث بالترتيب عن هذه functions:

on_add_shuttle_command_1

parse_add_shuttle_typed_command_2

add_shuttle_start_enabled_from_command_3

resolve_add_shuttle_request_4

create_managed_shuttle_5

finish_add_shuttle_6

request_spawn_if_needed_7

هذه السلسلة هي أفضل تمرين لفهم كيف يتحول command من نص في terminal إلى shuttle داخل simulation.

الفرق بين ADD topic و Control topic

من أكثر الأخطاء شيوعاً الخلط بين topic الإضافة و topic التحكم.

Commands	الغرض	Topic
----------	-------	-------

ADD_STOPPED, ADD_MOVING إنشاء shuttle جديد shuttles/add_command

ON, OFF, RESET, REMOVE shuttles/command التحكم بـ shuttle موجود

إذا أرسلت ON إلى add topic فهذا ليس المكان الصحيح. وإذا أرسلت ADD_STOPPED إلى control topic فهذا أيضاً غير صحيح.

تمرين 1: اكتشف النظام بالأوامر

شغل Room 315 ثم نفذ:

```
ros2 topic list | grep /room_315/rails
```

اختر add topic:

```
ros2 topic info /room_315/rails/right/shuttles/add_command
```

اعرض message:

```
ros2 interface show mfja_rail_interfaces/msg/ShuttleCommand
```

الهدف من التمرين: لا تعتمد على الحفظ. تعلم كيف تسأل ROS عن النظام.

تمرين 2: أضف Shuttle ساكن

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_10', command: 'ADD_STOPPED', start_slot: '2', speed: 0.2}"
```

راقب state

```
ros2 topic echo /room_315/rails/right/shuttles/state \
```

```
mfja_rail_interfaces/msg/ShuttleState
```

يجب أن ترى shuttle موجود لكنه ليس moving. بعدها شغله:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_10', command: 'ON'}"
```

تمرين 3: أضف Shuttle متحرك مباشرة

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_11', command: 'ADD_MOVING', start_slot: '3', speed: 0.2}"
```

راقب state وسترى أنه يتحرك مباشرة إذا لم يوجد blocker:

```
ros2 topic echo /room_315/rails/right/shuttles/state \
```

```
mfja_rail_interfaces/msg/ShuttleState
```

إذا تم رفض الأمر، غالباً start slot occupied. جرب slot آخر أو أزل shuttle.

تمرين 4: أوقف Shuttle ثم أعد تشغيله

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_11', command: 'OFF'}"
```

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_11', command: 'ON'}"
```

راقب كيف يتغير mode وموضع shuttle.

هذا يوضح الفرق بين object موجود لكنه disabled، وبين object محذوف.

تمرين 5: REMOVE و RESET

RESET يعيد shuttle إلى REMOVE start slot. يحذفه.

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_11', command: 'RESET'}"
```

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_11', command: 'REMOVE'}"
```

إذا كان spawn request أو delete request لا يزال قيد التنفيذ، قد يرفض node بعض العمليات مؤقتاً.

Rail System: كيف يفكر Shuttle بالمسار؟

ال shuttle لا يرى rail ك mesh في Gazebo. يرى rail ك graph في الذاكرة. كل segment له بداية ونهاية، وال

switch يقرر أي segment يأتي بعده.

مثال مبسط:

A14 -> A1 switch -> A1G or A1S

A23 -> A3 switch -> A3G or A3S

عندما يكون switch EXTERIOR يختار branch معين، وعندما يكون INTERIOR يختار branch آخر.

s و s_ratio: ما معناهما؟

لفهم sensors و devices، يجب فهم s و s_ratio.

المعنى

مصطلح

مسافة طولية على segment، عملياً بالمتر، من بداية segment حتى موضع

s

.shuttle

s_ratio نسبة من 0 إلى 1 على طول segment: $s_ratio = s / \text{segment.length}$

إذا كان $s_ratio=0.0$ فهو قريب من بداية segment. إذا 1.0 فهو قريب من نهايته.

العلاقة بينهما:

$s = s_ratio * \text{segment.length}$

$s_ratio = s / \text{segment.length}$

لذلك device YAML يستخدم segment و s_ratio لوضع sensor/stopper على المسار.

Switches: Command vs State

عند إرسال switch command، هذا يعني "أريد أن يتحرك switch". لكن actual state قد يتغير بعد delay.

`ros2 topic pub --once /room_315/rails/right/switches/command \`

`mfja_rail_interfaces/msg/SwitchCommand \`

`"{switches: [{name: 'A1', state: 'INTERIOR'}]}"`

راقب actual state:

`ros2 topic echo /room_315/rails/right/switches/state \`

`mfja_rail_interfaces/msg/SwitchState`

هذا الفصل بين command و state مهم في الأنظمة الحقيقية: request لا يعني أن الحركة انتهت فوراً.

Stoppers: لماذا ليست Switches؟

ال switch يغير الطريق. ال stopper يوقف shuttle مؤقتاً. كلاهما مرتبط بال rail، لكن وظيفتهما مختلفة.

العنصر	يغير route؟	يوقف shuttle؟
--------	-------------	---------------

Switch	نعم	لا بشكل مباشر
--------	-----	---------------

Stopper	لا	نعم
---------	----	-----

workflow الصحيح: نوقف shuttle بال stopper، نغير switch، ثم نفتح stopper.

Sensors: كيف تظهر القراءات؟

ال node يحسب sensor readings بناءً على موضع shuttle. إذا دخل shuttle منطقة sensor، ينشر

SensorReading داخل SensorFeedback.

SensorReading:

name

sensor_type

active

shuttle_name

segment

s

s_ratio

distance_m

هذا يعني أنك تستطيع debugging sensor بدون النظر إلى Gazebo: اقرأ topic فقط.

كيف أعدل مكان Sensor؟

الخطوات العملية:

1. حدد هل sensor على right أو left rail.

2. افتح rail_devices_right.yaml أو rail_devices_left.yaml.

3. ابحث عن name.

4. عدل segment و s_ratio.

5. شغل device validator، وهو script الفحص room_315_devices_validator.py.

6. أعد تشغيل Gazebo.

```
python3 src/mfja_3rd_floor_gz/mfja_robot_control_config/scripts/room_315_devices_validator.py
```

إذا أعطى SUMMARY PASS فملفات devices صحيحة من ناحية البنية. إذا أعطى FAIL اقرأ السطر الذي يبدأ ب FAIL:

لتعرف أي sensor أو stopper فيه خطأ.

كيف أستخدام Gazebo Coordinate؟

أحياناً تختار نقطة بصرياً في Gazebo وتريد تحويلها إلى rail device location . استخدم tool:

```
python3 src/mfja_3rd_floor_gz/mfja_robot_control_config/scripts/room_315_device_position_tool.py \
```

```
--side right \
```

```
--x -10.50 \
```

```
--y -3.20 \
```

```
--z 0.85
```

الأداة ترجع أقرب segment و s_ratio. بعدها تنسخ القيم إلى YAML.

كيف أضيف Feature جديدة؟ خريطة قرار

قبل كتابة الكود، اسأل:

1. هل feature هي command جديد على shuttle موجود؟ إذا نعم، وسع parser في control topic.

2. هل feature تضيف shuttle بطريقة جديدة؟ إذا نعم، وسع add command parser.

3. هل تحتاج state جديد؟ إذا نعم، أضف field أو topic state جديد.

4. هل تحتاج data جديدة من المستخدم؟ إذا نعم، هل تكفي fields الحالية؟

5. هل data الجديدة عامة ومتكررة؟ ربما تحتاج message جديد.

لا تبدأ بإضافة topic جديد دائماً. أحياناً command value جديد يكفي وأبسط.

تطبيق بيدك: تتبع ADD_STOPPED

لنفترض أنك تريد التأكد لماذا ADD_STOPPED يجعل shuttle ينتظر. أين تنظر في الكود؟

افتح function:

```
_add_shuttle_start_enabled_from_command
```

ستجد القرار النهائي واضحاً:

```
if command == 'ADD_STOPPED':
```

```
    return False
```

ثم شغل:

```
python3 -m py_compile mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
```

تطبيق بيدك: أضف Topic مراقبة جديد

مثال تعليمي: تريد topic ينشر عدد shuttles الحالي. الطريقة:

1. اختر message موجود: std_msgs/msg/Int32.

2. أضف import في Python.

3. أضف publisher في constructor.

4. في tick_ أو بعد add/remove, انشر العدد.

```
from std_msgs.msg import Int32
```

```
self.shuttle_count_publisher = self.create_publisher(
```

```
Int32,
```

```
f'/room_315/rails/{self.rail_side}/shuttles/count',
```

```
10,
```

```
)
```

```
msg = Int32()
```

```
msg.data = len(self.shuttles)
```

```
self.shuttle_count_publisher.publish(msg)
```

هذا مثال بسيط يوضح دورة إضافة `publisher`.

تطبيق بيدك: أضف Command جديد للتحكم

لنفترض تريد `command` اسمه `PAUSE` يعمل مثل `OFF`. لا تحتاج `topic` جديد. عدل `normalization`:

```
def _normalize_enabled_state(raw_state):
```

```
if state in {'0', 'OFF', 'DISABLE', 'DISABLED', 'STOP', 'PAUSE', 'FALSE'}:
```

```
    return False
```


إذا command جديد يعني action مختلف تماماً، عندها تعدل:

normalize_shuttle_action_•

apply_shuttle_action_•

behavior function جديدة تنفذ

تطبيق بيدك: أضيف Parameter جديد

إذا feature تحتاج رقم أو boolean قابل للتغيير من launch، استخدم parameter.

```
self.declare_parameter('my_feature_enabled', False)
```

```
self.my_feature_enabled = bool(
```

```
    self.get_parameter('my_feature_enabled').value
```

```
)
```

ثم في launch:

```
parameters=[
```

```
    common_parameters,
```

```
{
```

```
    'my_feature_enabled': LaunchConfiguration('my_feature_enabled'),
```

```
},
```

]

لا تضع constants داخل الكود إذا كنت تتوقع تغييرها أثناء التجارب.

تطبيق بيدك: أضيف Launch Argument

في launch file أضيف:

```
DeclareLaunchArgument(  
  
    'my_feature_enabled',  
  
    default_value='false',  
  
    choices=['true', 'false'],  
  
    description='Enable my feature.',  
  
)
```

ثم مرره إلى node parameters:

```
'my_feature_enabled': LaunchConfiguration('my_feature_enabled')
```

ثم تستخدمه:

```
ros2 launch mfja_room_315_bringup room_315_only.launch.py \
```

```
my_feature_enabled:=true
```

كيف تختبر بعد التعديل؟

1. افحص syntax:

```
python3 -m py_compile mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
```

2. ابن workspace:

```
cd /home/tiago/ALI_ros2_ws
```

```
source /opt/ros/jazzy/setup.bash
```

```
colcon build --symlink-install
```

```
source install/setup.bash
```

3. شغل launch.

4. افحص topics:

```
ros2 topic list | grep room_315
```

5. أرسل command صغير.

6. راقب state topic و logs.

قراءة Logs وفهم الأخطاء

الكود يستعمل logger لرسائل مهمة مثل:

• Failed to add shuttle

start slot is occupied•

Gazebo spawn service is not ready yet•

Unknown shuttle•

entered FALLING mode•

لا تتجاهل logs. غالباً تخبرك مباشرة أين المشكلة: topic خطأ، command غير مدعوم، slot مشغول، service غير جاهز، أو route غير صالح.

DISABLED, WAITING, FALLING

Mode/State	متى يحدث؟	ماذا تفعل؟
WAITING	shuttle متوقف، بسبب OFF أو stopper أو collision	افحص stopped_by أو شغل .ON
DISABLED	أمر OFF أو ADD_STOPPED	أرسل .ON
FALLING	لا يوجد successor صالح حسب switch routing	صحح switch ثم RESET

في debugging، لا تسأل "لماذا لا يتحرك؟" فقط. اسأل: ما هو mode؟ ما هو stopped_by؟ هل speed أكبر من صفر؟

كيف تعمل Occupied Start Slot؟

عند إضافة node، shuttle يفحص إذا كان start slot مشغولاً. إذا كان shuttle آخر قريباً من نفس slot ضمن radius محدد، يرفض الإضافة.

```
reject_occupied_start_slots = true
```

start_slot_occupancy_radius_m = 0.33

هذا يمنع spawn shuttles فوق بعضها. إذا تريد التجربة من slot معين، تأكد أنه فارغ أو استخدم REMOVE أو انتظر حتى shuttle بيتعد.

Left Rail و Right Rail: نفس المنطق، Config مختلف

لدينا node لنفس الكود لكنه يعمل مرة على right rail ومرة على left rail. الفرق يأتي من parameters مثل:

• rail_side:=right أو left

• network_yaml

• devices_yaml

• topic_defaults

هذا تصميم جيد لأننا لا نكرر الكود. نفس class يخدم الجهتين.

لماذا نعلم Topics نهائية فقط؟

أثناء التطوير تغيرت أسماء topics أكثر من مرة، لكن النسخة النهائية أبسط: نعلم فقط على namespace واحد واضح هو /

{right,left}/rails/room_315/.... لذلك لا توجد aliases قديمة في الكود النهائي.

Topic النهائي	النوع
room_315/rails/right/shuttles// add_command	إضافة shuttle
room_315/rails/right/switches/command/	تحكم switches

النوع	Topic النهائي
-------	---------------

room_315/rails/right/stoppers/command/ تحكم stoppers

إذا بدك left rail، غيّر right إلى left بنفس البنية.

كيف تربط Robot مع Shuttle؟

robots في المشروع تستخدم topics مختلفة مثل /kuka1/joint_trajectory. ال shuttle يستخدم rail topics.

كلاهما موجودان في Gazebo world نفسه.

الفكرة:

• Robot يتحرك عبر joint trajectory.

• Shuttle يتحرك عبر set_pose kinematic.

• Gazebo يعرضهما في نفس scene.

• Collision models يمكن أن تتفاعل بصرياً/فيزيائياً حسب masks.

لذلك robot-shuttle experiments تحتاج تشغيل world مع robots و shuttles معاً.

متى أستخدم Room 315 ومتى Full Floor؟

الوضع	استخدمه عندما...
-------	------------------

Room 315 only تريد اختبار rails/shuttles/sensors بسرعة.

Full floor تريد البيئة الكاملة أو robots أخرى.

الوضع	استخدمه عندما...
-------	------------------

Single industrial robot تريد اختبار robot فقط بدون Room 315.

ابدأ دائماً بأصغر launch يكتفيك. هذا يجعل debugging أسرع.

كيف تقرأ Launch File؟

ابحث عن ثلاثة أشياء:

1. DeclareLaunchArgument: ما هي الخيارات المتاحة؟

2. IncludeLaunchDescription: أي launch آخر يتم تشغيله؟

3. Node: ما هي nodes التي تبدأ؟ وما هي parameters؟

مثال argument:

```
DeclareLaunchArgument(
    'room315_shuttles_start_enabled',

    default_value='false',

    choices=['true', 'false'],

)
```

هذا يعني أنك تستطيع تمريره من terminal.

كيف تربط Launch Argument مع Node Parameter؟

في argument, launch, لا يؤثر على node حتى تمرره داخل parameters.

```
'start_enabled': LaunchConfiguration('room315_shuttles_start_enabled')
```

ثم داخل node:

```
start_enabled = bool(self.get_parameter('start_enabled').value)
```

هذا هو الجسر بين terminal argument والكود الداخلي.

كيف أضيف حالة جديدة لل Shuttle؟

إذا أردت mode جديد مثل CHARGING مثلاً، ستحتاج:

1. تعريف constant جديد في الكود.

2. تحديد متى يدخل shuttle هذا mode.

3. تحديد ماذا يفعل tick loop في هذا mode.

4. تحديث state publication حتى يظهر mode.

5. تحديث README/report.

لا تضيف mode جديد فقط لأنه اسم جميل. أضفه عندما يكون له behavior مختلف فعلاً.

كيف أضيف Sensor Type جديد؟

1. أضف category أو field في device YAML إذا لزم.

2. عدّل loader الذي يقرأ devices.

3. أضف logic يحسب متى sensor active.

4. انشر SensorReading مناسباً.

5. أضف marker style إذا تريد رؤيته في Gazebo.

إذا sensor الجديد مجرد position sensor بنمط مختلف، ربما يكفي YAML فقط.

كيف أضيف Switch جديد؟

هذا أصعب من إضافة command لأنه مرتبط بالrail graph.

1. أضف geometry/segments المطلوبة.

2. حدث rail_network_*.yaml.

3. حدث switch mapping وال public labels إذا لزم.

4. حدث device YAML لل stoppers/sensors.

5. شغل validators.

```
ros2 run mfja_robot_control_config room_315_network_validator.py
```

```
ros2 run mfja_robot_control_config room_315_continuous_path_validator.py
```

كيف أضيف Start Slot جديد؟

Start slots تأتي من device/network configuration. يجب أن تكون على rail segment صالح.

1. حدد موقع slot على rail.

2. احسب segment و s_ratio.

3. أضفه في device YAML أو network config حسب التصميم الحالي.

4. تأكد أن add command يقبل هذا slot.

5. اختبر occupied slot logic.

لا تضيف slot في Gazebo بصرياً فقط؛ يجب أن يعرفه kinematic core.

كيف أفهم أخطاء Build؟

نوع الخطأ	غالباً سببه
Python syntax error	خطأ indentation أو أقواس. استخدم py_compile.
Message import error	لم تبين mfja_rail_interfaces أو لم تعمل source.
Launch argument unknown	أضفت argument في launch مختلف عن الذي تشغله.
Topic لا يظهر	node لم يبدأ أو namespace مختلف.

منهجية Debug لأي Feature جديدة

1. هل الأمر يصل؟ ضع log في callback.

2. هل parser يفهمه؟ اطبع normalized action.

3. هل state الداخلي تغير؟ راقب state topic.

4. هل Gazebo service جاهز؟ افحص service list.

5. هل model موجود؟ راقب gazebo scene/logs.

6. هل documentation مطابق للكود؟ جرب copy-paste من HTML.

لا تقفز مباشرة إلى gazebo visual. ابدأ من topic ثم callback ثم state ثم gazebo.

تمرين شامل: افهم مسار ADD_STOPPED بنفسك

المطلوب: تتبع كيف يصل أمر ADD_STOPPED من terminal إلى إنشاء shuttle ساكن.

1. افتح room_315_kinematic_shuttle_node.py.

2. ابحث عن on_add_shuttle_command.

3. انتقل إلى parse_add_shuttle_typed_command.

4. لاحظ أن ADD_STOPPED تعطي enabled=False.

5. شغل Room 315.

6. أرسل add command بالقيمة النهائية.

7. راقب أن shuttle ينتظر ولا يتحرك.

```
ros2 topic pub --once /room_315/rails/right/shuttles/add_command \
```

```
mfja_rail_interfaces/msg/ShuttleCommand \
```

```
"{name: 'room315_right_shuttle_test', command: 'ADD_STOPPED', start_slot: '2', speed: 0.2}"
```

تمرين شامل: أضف Documentation صحيح

بعد أي feature، لا تتركها في الكود فقط. وثقها في:

README.md•

runbook.html•

• هذا التقرير إذا كانت feature تعليمية مهمة

اكتب دائماً:

1. ما المشكلة التي تحلها؟

2. ما الأمر الجديد؟

3. مثال copy-paste.

4. هل يوجد تأثير على commands الحالية؟

ملخص العلاقات بين الملفات

mfja_rail_interfaces/msg room_315_kinematic_shuttle_node.py Gazebo services launch files

YAML config

اقرأ الرسم هكذا: messages تحدد شكل البيانات، launch يشغل node ويمرر YAML، parameters يعطي

geometry/devices، وال node يربط كل ذلك مع Gazebo.

مصطلحات يجب أن تحفظها

مصطلح	معنى سريع في مشروعنا
Node	برنامج يتحكم أو ينشر state.
Topic	قناة رسائل.

Message	شكل البيانات على topic.
Callback	function تعمل عند وصول message.
Parameter	إعداد node.
Launch Argument	إعداد تمرره عند تشغيل launch.
Service Client	node يطلب عملية من service مثل Gazebo create.
Kinematic	حركة محسوبة مباشرة، لا تعتمد على physics wheels.

خطة مراجعة قبل الامتحان أو المناقشة

1. اشرح لماذا اخترنا kinematic simulation.
2. ارسم Topic flow من terminal إلى node.
3. اذكر الفرق بين add topic و control topic.
4. اشرح ADD_STOPPED مقابل ADD_MOVING.
5. اشرح switch command/state delay.
6. اشرح stopper workflow.
7. اشرح كيف تعمل sensors بدون physical Gazebo sensor.
8. افتح الكود واعرض callback الذي يستقبل add command.
9. اعرض أين تضيف alias جديد.

الخلاصة الكبرى

إذا أردت فهم المشروع بعمق، اختصره في هذه الجملة:

لدينا ROS 2 Node يقرأ أوامر typed من Topics، يحولها إلى state داخلي لا Shuttles على rail graph، ثم

يحرك Gazebo models عبر services وينشر state/sensor feedback.

كل feature مستقبلية ستدور حول هذه الحلقة. عندما تضيف command جديداً، اسأل: أين يدخل؟ كيف أفسره؟ أي state

يغير؟ ماذا أنشر بعده؟ وهل يجب أن يظهر في Gazebo؟

إذا استطعت الإجابة على هذه الأسئلة، فأنت لا تستخدم المشروع فقط؛ أنت أصبحت قادراً على تطويره بيدك.